

Chiang Mai University

AnyaSpyAC

Suthana Kwaonueng, Theerada Siri, Sorawach Utas

2026-11-10

1 Contest 2
2 Tanya 2
3 cellul4r 13
4 Ohm 21
5 Mathematics 21

Contest (1)

template.cpp

55 lines

```cpp
#include <bits/stdc++.h>
using namespace std;

// ---- User Defined Types & Shorthands ----
using ll = long long;
using ld = long double;
using cd = complex<ld>;

using pi = pair<int, int>;
using pl = pair<ll, ll>;
using pd = pair<ld, ld>;

using vi = vector<int>;
using vd = vector<ld>;
using vl = vector<ll>;
using vpi = vector<pi>;
using vpl = vector<pl>;
using vcd = vector<cd>;

template<class T> using pq = priority_queue<T>;
template<class T> using pqg = priority_queue<T, vector<T>,
    greater<T>>;

// ---- Macros & Utilities ----
#define rep(i, a) for (int i = 0; i < a; ++i)
#define FOR(i, a, b) for (int i = a; i < (b); i++)
#define F0R(i, a) for (int i = 0; i < (a); i++)
#define FORd(i, a, b) for (int i = (b)-1; i >= a; i--)
#define F0Rd(i, a) for (int i = (a)-1; i >= 0; i--)
#define trav(a, x) for (auto& a : x)
#define uid(a, b) uniform_int_distribution<long long>(a, b)(rng
    )

#define sz(x) (int)(x).size()
#define mp make_pair
#define pb push_back
#define lb lower_bound
#define ub upper_bound
#define all(x) x.begin(), x.end()
#define ins insert

template<class T> bool ckmin(T& a, const T& b) { return b < a ?
     a = b, 1 : 0; }
template<class T> bool ckmax(T& a, const T& b) { return a < b ?
     a = b, 1 : 0; }

// ---- RNG Seed ----
mt19937 rng(chrono::steady_clock::now().time_since_epoch().
    count());

// ---- Constants ----
const char nl = '\n';
const int N = 2e5+3;
const int MOD = 998244353;
const int INF = INT_MAX;
const long long LINF = LLONG_MAX;

int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
}
```

.vimrc

21 lines

```
"General editor settings
set tabstop=4
set nocompatible
set shiftwidth=4
set expandtab
set autoindent
set smartindent
set ruler
set showcmd
set incsearch
set shellslash
set number
set relativenumber
set cino+=L0

"keybindings for { completion, "jk" for escape, ctrl-a to
    select all
inoremap {<CR>  {<CR>}<Esc>O
inoremap {}     {}
imap jk         <Esc>
map <C-a> <esc>ggVG<CR>
set belloff=all
```

.bashrc

1 lines

```
export PATH=$PATH:~/scripts/
```

build.sh

1 lines

```
g++ -static -DLOCAL -lm -s -x c++ -Wall -Wextra -O2 -std=c++17
    -o $1 $1.cpp
```

stress.sh

22 lines

```
#!/usr/bin/env bash

for ((testNum=0;testNum<$4;testNum++))
do
    ./$3 > input
    ./$2 < input > outSlow
    ./$1 < input > outWrong
    H1=`md5sum outWrong`
    H2=`md5sum outSlow`
    if !(cmp -s "outWrong" "outSlow")
    then
        echo "Error found!"
        echo "Input:"
        cat input
        echo "Wrong Output:"
        cat outWrong
        echo "Slow Output:"
        cat outSlow
        exit
    fi
done
echo Passed $4 tests
```

Tanya (2)

2.1 Template & Utilities

debug.h

Description: will show debug only when add -DDEBUG to the compile flag, e.g. g++ -DDEBUG -o file file.cpp

d394c1, 28 lines

```cpp
void __print(int x) {cerr << x;}
void __print(long x) {cerr << x;}
void __print(long long x) {cerr << x;}
void __print(unsigned x) {cerr << x;}
void __print(unsigned long x) {cerr << x;}
void __print(unsigned long long x) {cerr << x;}
void __print(float x) {cerr << x;}
void __print(double x) {cerr << x;}
void __print(long double x) {cerr << x;}
void __print(char x) {cerr << '\'' << x << '\'';}
void __print(const char *x) {cerr << '\"' << x << '\"';}
void __print(const string &x) {cerr << '\"' << x << '\"';}
void __print(bool x) {cerr << (x ? "true" : "false");}

template<typename T, typename V>
void __print(const pair<T, V> &x);
template<typename T>
void __print(const T &x) {int f = 0; cerr << '{'; for (auto &i:
     x) cerr << (f++ ? ", " : ""), __print(i); cerr << "}";}
template<typename T, typename V>
void __print(const pair<T, V> &x) {cerr << '{'; __print(x.first
    ); cerr << ", "; __print(x.second); cerr << '}';}
void _print() {cerr << "]\n";}
template <typename T, typename... V>
void _print(T t, V... v) {__print(t); if (sizeof...(v)) cerr <<
     ", "; _print(v...);}
#ifdef DEBUG
#define dbg(x...) cerr << "\e[91m"<<__func__<<":"<<__LINE__<<"
    [" << #x << "] = ["; _print(x); cerr << "\e[39m" << endl;
#else
#define dbg(x...)
#endif
```

pragma.h

Description: random pragma magic that I don't fucking know how this work, but it ocassionaly solve my stupid bad implementation code in the past(segtree constant factor), this should not be mandatory and only last resort

1021ce, 7 lines

```cpp
//some one on cf told us that these are mostly enough
#pragma GCC target ("avx2")
#pragma GCC optimize ("O3")
#pragma GCC optimize ("unroll-loops")
//just in case have this one
#pragma GCC optimize("O3,unroll-loops")
#pragma GCC target("avx2,bmi,bmi2,popcnt,lzcnt")
```

2.2 Number Theory & Combinatoric

factorial.h

Description: use 2*N when do stars and bars

061239, 20 lines

```cpp
long long fac[N], faci[N];

long long bigmod(long long a, long long n){
    long long res = 1;
    while(n>0){
        if(n&1){
            res = res * a % MOD;
        }
        a = a * a % MOD;
        n>>=1;
```

```
    }
    return res;
}

void cal_fac(){
    fac[0] = 1;
    for(int i=1;i<N;++i) fac[i] = fac[i-1] * i % MOD;
    faci[N-1] = bigmod(fac[N-1], MOD-2);
    for(int i=N-2;i>=0;--i) faci[i] = faci[i+1] * (i+1) % MOD;
}
```

Cnr.h

Description: this is ok

cc78ad, 11 lines

```
long long Cnr(int n, int k) {
    if(k<0 || n<k)return 0;
    if(k==0 or n==k)return 1;
    /*
    double res = 1;
    for (int i = 1; i <= k; ++i)
        res = res * (n - k + i) / i;
    return (long long)(res + 0.01);
    */
    return fac[n] * faci[k] % MOD * faci[n-k] % MOD;
}
```

lucas.h

Description: for large n and small modulo, the complexity is O(p * log-p(n)), p must be prime

10284e, 36 lines

```
// Standard nCr modulo p for small n and r
long long nCr_small(long long n, long long r, int p) {
    if (r < 0 || r > n) return 0;
    if (r == 0 || r == n) return 1;
    if (r > n / 2) r = n - r;

    long long num = 1, den = 1;
    for (int i = 0; i < r; ++i) {
        num = (num * (n - i)) % p;
        den = (den * (i + 1)) % p;
    }

    // Modular inverse using Fermat's Little Theorem (den^(p-2)
        % p)
    auto power = [&](long long base, long long exp) {
        long long res = 1;
        base %= p;
        while (exp > 0) {
            if (exp % 2 == 1) res = (res * base) % p;
            base = (base * base) % p;
            exp /= 2;
        }
        return res;
    };

    return (num * power(den, p - 2)) % p;
}

// Lucas Theorem Implementation
long long lucas(long long n, long long k, int p) {
    // Base cases
    if (k == 0) return 1;
    if (n < k) return 0;

    // Recursive step: nCr % p = ( (n/p)C(k/p) * (n%p)C(k%p) )
        % p
    return (lucas(n / p, k / p, p) * nCr_small(n % p, k % p, p)
        ) % p;
}
```

factoradix.h

Description: for that factorial base number finding nth permutation blablabla, also this using 0 base indexing, i.e. your 1st lexicographically permutation now is start at 0th

<ext/pb_ds/assoc_container.hpp>, <ext/pb_ds/tree_policy.hpp> e835c3, 57 lines

```
typedef __gnu_pbds::tree<int, __gnu_pbds::null_type, less<int>,
     __gnu_pbds::rb_tree_tag, __gnu_pbds::
    tree_order_statistics_node_update> ordered_set;

ordered_set st;

vector<int> factoradix_to_permutation(int n, vector<int> &
    factoradix){
    st.clear(); for(int i=1;i<=n;++i) st.ins(i);
    int cl = 0;
    while((int)factoradix.size() < n){
        factoradix.pb(0); ++cl;
    }

    vector<int> res;
    for(int i=(int)factoradix.size()-1;i>=0;--i){
        auto it = st.find_by_order(factoradix[i]);
        res.pb(*it);
        st.erase(it);
    }

    while(cl--) factoradix.pop_back();

    return res;
}

vector<int> permutation_to_factoradix(int n, vector<int> &
    permutation){
    st.clear(); for(int i=1;i<=n;++i) st.ins(i);
    vector<int> res;
    for(int i=0;i<(int)permutation.size();++i){
        int cnt = st.order_of_key(permutation[i]);
        res.pb(cnt);
        auto it = st.find_by_order(cnt); st.erase(it);
    }
    reverse(all(res));
    while(!res.empty() and res.back() == 0)res.pop_back();

    return res;
}

vector<int> decimal_to_factoradix(ll n){
    vector<int> res;
    int d = 1;
    while(n > 0){
        res.pb(n % d);
        n/=d; ++d;
    }
    return res;
}

ll factoradix_to_decimal(vector<int> &cur){
    ll res = 0;
    ll d = 1;
    for(int i=0;i<(int)cur.size();++i){
        res += d*cur[i];
        d*=(i+1);
    }
    return res;
}
```

div-ceil-floor.h

Description: to stop c++ fucked up negative case of ceil, floor

2834e8, 9 lines

```
long long div_floor(__int128 a, long long b){
    __int128 q = a / b;
    __int128 r = a % b;
    if (r != 0 && ((a < 0) != (b < 0))) --q;
    return (long long)q;
}
long long div_ceil(__int128 a, long long b){
    return -div_floor(-a, b);
}
```

sieve.h

Description: construct is O(sqrt(n)), can use to find all prime factor of a number in O(log n)

b28bf2, 10 lines

```
vector<bool> is_prime(M+1, true);
void sieve(){
    is_prime[0] = is_prime[1] = false;
    for (int i = 2; i <= M; i++) {
        if (is_prime[i]) {
            for (ll j = (ll)i * i; j <= M; j += i)
                is_prime[j] = false;
        }
    }
}
```

linear-sieve.h

Description: lp[i] = minimum prime that divide i

8c3e32, 17 lines

```
vector<int> lp(N+1);
vector<int> pr;

void sieve(){
    for (int i=2; i <= N; ++i) {
        if (lp[i] == 0) {
            lp[i] = i;
            pr.push_back(i);
        }
        for (int j = 0; i * pr[j] <= N; ++j) {
            lp[i * pr[j]] = pr[j];
            if (pr[j] == lp[i]) {
                break;
            }
        }
    }
}
```

phi-sieve.h

b11905, 13 lines

```
vector<int> phi(M+1);

void phi_1_to_n() {
    for (int i = 0; i <= M; i++)
        phi[i] = i;

    for (int i = 2; i <= M; i++) {
        if (phi[i] == i) {
            for (int j = i; j <= M; j += i)
                phi[j] -= phi[j] / i;
        }
    }
}
```

phi-function.h

fb8d57, 13 lines

```
int phi(int n) {
    int result = n;
    for (int i = 2; i * i <= n; i++) {
```

```
        if (n % i == 0) {
            while (n % i == 0)
                n /= i;
            result -= result / i;
        }
    }
    if (n > 1)
        result -= result / n;
    return result;
}
```

mobius-function.h

c6a8f4, 10 lines

```
int mob[N+1];
void mobius() {
  mob[1] = 1;
  for (int i = 2; i <= N; i++){
    mob[i]--;
    for (int j = i + i; j <= N; j += i) {
      mob[j] -= mob[i];
    }
  }
}
```

miller-rabin.h

Description: randomize primality test O(logn**2)

b45121, 52 lines

```
using u64 = uint64_t;
using u128 = __uint128_t;

u64 bigmod(u64 base, u64 e, u64 mod){
    u64 result = 1;
    base %= mod;
    while(e){
        if(e & 1)
            result = (u128)result * base % mod;
        base = (u128)base * base % mod;
        e >>= 1;
    }
    return result;
}

bool check_composite(u64 n, u64 a, u64 d, int s){
    u64 x = bigmod(a, d, n);
    if(x == 1 || x == n - 1)
        return false;
    for(int r = 1; r < s; r++){
        x = (u128)x * x % n;
        if(x == n - 1)
            return false;
    }
    return true;
}

//random number generator
mt19937 rng(chrono::steady_clock::now().time_since_epoch().
    count());
long long rnd(long long x, long long y) {
  return uniform_int_distribution<long long>(x, y)(rng);
}

// return true if n is probably prime, else return false.
bool MillerRabin(u64 n, int iter=5){
    if(n < 4){
        return n == 2 || n == 3;
    }
    int s = 0;
    u64 d = n-1;
    while((d & 1) == 0){
        d >>= 1;
        s++;
    }

    for(int i = 0; i < iter; i++){
        long long a = rnd(2, n-2);
        if(check_composite(n, a, d, s))
            return false;
    }
    return true;
}
```

pollard-rho.h

Description: randomize factorization in O(n**1/4 * logn)

d08505, 22 lines

```
#define uid(a, b) uniform_int_distribution<long long>(a, b)(rng
    )
 mt19937 rng(chrono::steady_clock::now().time_since_epoch().
     count());

long long mult(long long a, long long b, long long mod) {
    return (__int128)a * b % mod;
}

long long f(long long x, long long c, long long mod) {
    return (mult(x, x, mod) + c) % mod;
}

long long rho(long long n, long long x0=2, long long c=1) {
    long long x = x0, y = x0, g = 1;
    while (g == 1) {
        x = f(x, c, n);
        y = f(f(y, c, n), c, n);
        g = __gcd(abs(x - y), n);
    }

    if(g == n) return rho(n, uid(2, n-1), uid(-2, 2)); //uid is
        uniform long
    else return g;
}
```

discrete-log.h

Description: Returns minimum x for which a**x === b (mod m)

f7650d, 32 lines

```
int solve(int a, int b, int m) {
    a %= m, b %= m;
    int k = 1, add = 0, g;
    while ((g = __gcd(a, m)) > 1) {
        if (b == k)
            return add;
        if (b % g)
            return -1;
        b /= g, m /= g, ++add;
        k = (k * 1ll * a / g) % m;
    }

    int n = sqrt(m) + 1;
    int an = 1;
    for (int i = 0; i < n; ++i)
        an = (an * 1ll * a) % m;

    map<int, int> vals;
    for (int q = 0, cur = b; q <= n; ++q) {
        vals[cur] = q;
        cur = (cur * 1ll * a) % m;
    }

    for (int p = 1, cur = k; p <= n; ++p) {
        cur = (cur * 1ll * an) % m;
        if (vals.count(cur)) {
            int ans = n * p - vals[cur] + add;
            return ans;
        }
    }
    return -1;
}
```

primitive-root.h

Description: The following code assumes that the modulo p is a prime number. To make it works for any value of p, we must add calculation of phi(p)

c6d472, 30 lines

```
int powmod (int a, int b, int p) {
    int res = 1;
    while (b)
        if (b & 1)
            res = int (res * 1ll * a % p),  --b;
        else
            a = int (a * 1ll * a % p),  b >>= 1;
    return res;
}

int generator (int p) {
    vector<int> fact;
    int phi = p-1,  n = phi; // cal phi p by assuming it's
        prime
    for (int i=2; i*i<=n; ++i)
        if (n % i == 0) {
            fact.push_back (i);
            while (n % i == 0)
                n /= i;
        }
    if (n > 1)
        fact.push_back (n);

    for (int res=2; res<=p; ++res) {
        bool ok = true;
        for (size_t i=0; i<fact.size() && ok; ++i)
            ok &= powmod (res, phi / fact[i], p) != 1;
        if (ok)  return res;
    }
    return -1;
}
```

diophantine.h

4fcad2, 89 lines

```
ll gcd(ll a, ll b, ll& x, ll& y) { //gcd extended
    if (b == 0) {
        x = 1;
        y = 0;
        return a;
    }
    ll x1, y1;
    ll d = gcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - y1 * (a / b);
    return d;
}

bool find_any_solution(ll a, ll b, ll c, ll &x0, ll &y0, ll &g)
    {
    g = gcd(abs(a), abs(b), x0, y0);
    if (c % g) {
        return false;
    }

    x0 *= c / g;
    y0 *= c / g;
```

```
    if (a < 0) x0 = -x0;
    if (b < 0) y0 = -y0;
    return true;
}

void shift_solution(ll & x, ll & y, ll a, ll b, ll cnt) { //
     make sure that a,b coprime
    x += cnt * b;
    y -= cnt * a;
}

ll find_all_solutions(ll a, ll b, ll c, ll minx, ll maxx, ll
     miny, ll maxy) {
    ll x, y, g;
    if (!find_any_solution(a, b, c, x, y, g))
        return 0;
    a /= g;
    b /= g;

    ll sign_a = a > 0 ? +1 : -1;
    ll sign_b = b > 0 ? +1 : -1;

    shift_solution(x, y, a, b, (minx - x) / b);
    if (x < minx)
        shift_solution(x, y, a, b, sign_b);
    if (x > maxx)
        return 0;
    ll lx1 = x;

    shift_solution(x, y, a, b, (maxx - x) / b);
    if (x > maxx)
        shift_solution(x, y, a, b, -sign_b);
    ll rx1 = x;

    shift_solution(x, y, a, b, -(miny - y) / a);
    if (y < miny)
        shift_solution(x, y, a, b, -sign_a);
    if (y > maxy)
        return 0;
    ll lx2 = x;

    shift_solution(x, y, a, b, -(maxy - y) / a);
    if (y > maxy)
        shift_solution(x, y, a, b, sign_a);
    ll rx2 = x;

    if (lx2 > rx2)
        swap(lx2, rx2);

    ll lx = max(lx1, lx2);
    ll rx = min(rx1, rx2);
    if (lx > rx)
        return 0;

    return (rx - lx) / abs(b) + 1;
}
//number of solution of ax + by = c in general
ll number_of_solution(ll a, ll b, ll c, ll x1, ll x2, ll y1, ll
      y2){
    if(a == 0 and b == 0 and c == 0)return (x2-x1+1)*(y2-y1+1);
    else if(a == 0 and b == 0) return 0;
    else if(a == 0){
        if(c%b!=0 or y1>c/b or y2<c/b)return 0;
        else return (x2-x1+1);
    } else if(b == 0){
        if(c%a!=0 or x1>c/a or x2<c/a)return 0;
        else return (y2-y1+1);
    } else {
        return find_all_solutions(a, b, c, x1, x2, y1, y2);
    }
}
```

crt.h

efd085, 51 lines

```
class CRT { // ChineseRemainderTheorem
    typedef long long vlong;
    typedef pair<vlong,vlong> pll;
    vector<pll> equations;

public:
    void clear() {
        equations.clear();
    }
    void addEquation( vlong r, vlong m ) {
        equations.push_back({r, m});
    }

    void ext_gcd(vlong a, vlong b, vlong& x, vlong& y) {
        if (b == 0) {
            x = 1;
            y = 0;
            return;
        }
        vlong x1, y1;
        ext_gcd(b, a % b, x1, y1);
        x = y1;
        y = x1 - y1 * (a / b);
    }

    pll solve() {
        if (equations.size() == 0) return {-1,-1};

        vlong a1 = equations[0].first;
        vlong m1 = equations[0].second;
        a1 %= m1;
        for ( int i = 1; i < (int)equations.size(); i++ ) {
            vlong a2 = equations[i].first;
            vlong m2 = equations[i].second;

            vlong g = __gcd(m1, m2);
            if ( a1 % g != a2 % g ) return {-1,-1};
            vlong p, q;
            ext_gcd(m1/g, m2/g, p, q);

            vlong mod = m1 / g * m2;
            p = (p%mod+mod)%mod, q = (q%mod+mod)%mod;
            vlong x = ( (__int128)a1 * (m2/g) % mod *q % mod +
                 (__int128)a2 * (m1/g) % mod * p % mod ) % mod;
            a1 = x;
            if ( a1 < 0 ) a1 += mod;
            m1 = mod;
        }
        return {a1, m1};
    }
};
```

2.3 General Data Structures & Algorithms

order-statistics-tree.h

<ext/pb_ds/assoc_container.hpp> 9efa3e, 9 lines

```
using namespace __gnu_pbds;
template <typename T>
using ordered_set =
    tree<T, null_type, less<T>, rb_tree_tag,
         tree_order_statistics_node_update>;
ordered_set<int> st;
//st.order_of_key(x) - find # of elements in st stricly less
     than x, i.e. index of x
//st.size() - size of st
//st.find_by_order(x) - return iterator point to xth indexed
     element
//st.clear() - clear container
```

fenwick-tree.h

Description: use 1 base indexing

3185d9, 25 lines

```
vector<long long>bit;

void add(int i, int k){
    while(i<(int)bit.size()){
        bit[i]+=k;
        i+=i&-i;
    }
}

ll F(int i){
    ll sum=0;
    while(i>0){
        sum+=bit[i];
        i-=i&-i;
    }
    return sum;
}

void build_bit(vl &a){
    bit=a;
    for(int i=1;i<(int)bit.size();++i){
        int p=i+(i&-i);//index to parent
        if(p<(int)bit.size())bit[p]+=bit[i];
    }
}
```

dsu.h

Description: O(n) amortized by inverse ackerman function

c86435, 23 lines

```
int parent[N], _rank[N];

int find_set(int v) {
    if (v == parent[v])
        return v;
    return parent[v] = find_set(parent[v]);
}

void make_set(int v) {
    parent[v] = v;
    _rank[v] = 1;
}

void union_sets(int a, int b) {
    a = find_set(a);
    b = find_set(b);
    if (a != b) {
        if (_rank[a] < _rank[b])
            swap(a, b);
        parent[b] = a;
        _rank[a]+=_rank[b];
    }
}
```

dijkstra.h

Description: 0/1 bfs can be done using deque

3fd08c, 17 lines

```
//initialize dist, proc
    for (int i = 1; i <= m; i++) dist[i] = INF;
    dist[x] = 0;
```

```
    priority_queue<pair<ll, int>>q;
    q.push({0,x});
    while (!q.empty()) {
        int a = q.top().second; q.pop();
        if (proc[a]) continue;
        proc[a] = true;
        for (auto u : adj[a]) {
            int b = u.first, w = u.second;
            if (dist[a]+w < dist[b]) {
                dist[b] = dist[a]+w;
                q.push({-dist[b],b});
            }
        }
    }
```

floyd-warshall-stp.h

Description: before k-th phase d[i][j] = stp len from i to j using only vertices from set 1,2,...k-1 as internal path, the beginning and end of path are not restricted by this property.

ab1dcf, 10 lines

```
//don't forget to initialize the base case of weight
//this work for negative weight without negative cycle
for (int k = 0; k < n; ++k) {
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            if (d[i][k] < INF && d[k][j] < INF)
                d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
        }
    }
}
```

digit-dp-example.h

d01123, 31 lines

```
ll dp[20][11][2][2];
vector<int> num;

ll construct(int pos, int d, int flag, bool isStarted){
    if(pos == (int)num.size()){
        //return isStarted;
        return 1;
    }

    if(dp[pos][d][flag][isStarted] != -1)return dp[pos][d][flag
         ][isStarted];

    int LMT;
    if(flag == 0){
        LMT = num[pos];
    } else {
        LMT = 9;
    }

    //placing current digits
    ll res = 0;
    for(int i=0;i<=LMT;++i){
        if((i == 0 and isStarted and i == d) or (i != 0 and i
             == d))continue;
        //by the condition adjacent digits must not be the same
        int nflag = flag;
        if(i < LMT and flag == 0)nflag = 1;

        res += construct(pos+1, i, nflag, (isStarted or i>0));
    }

    return dp[pos][d][flag][isStarted] = res;
}
```

LIS-example.h

Description: this can also be use to find minimum partition on posit by finding maximum anti-chain

<algorithm>, <iostream>, <vector>

9406b2, 24 lines

```
using namespace std;

int main() {
    int n;
    cin >> n;
    vector<int> x(n);
    for (int i = 0; i < n; ++i) cin >> x[i];

    vector<int> best(n + 1);
    best[0] = 0;
    int result = 0;
    for (int i = 0; i < n; ++i) {
        int len = lower_bound(best.begin(), best.begin() +
             result + 1, x[i]) -
                  best.begin();
        if (len > result) {
            result = len;
            best[len] = x[i];
        } else {
            best[len] = min(best[len], x[i]);
        }
    }

    cout << result << endl;
}
```

lca.h

Description: I can implement simpler one than this, also don't forget to initialize adj

0ae36f, 35 lines

```
const int LOG = 20;
int dep[N], parent[N];
int up[N][LOG]; // 2^j-th ancestor of n

void dfs(int a,int e){
    for(auto b:adj[a]){
        if(b == e)continue;
        dep[b] = dep[a] + 1;
        parent[b] = a;
        up[b][0] = parent[b];
        for(int i=1;i<LOG;++i){
            up[b][i] = up[up[b][i-1]][i-1];
        }
        dfs(b,a);
    }
}

int lca(int a, int b){
    if(dep[a]<dep[b])swap(a,b);
    int k = dep[a] - dep[b];
    for(int i=LOG-1;i>=0;--i){
        if(k &(1<<i)){
            a = up[a][i];
        }
    }

    if(a == b)return a;
    for(int i=LOG-1;i>=0;--i){
        if(up[a][i] != up[b][i]){
            a = up[a][i];
            b = up[b][i];
        }
    }
    return up[a][0];
}
```

kth-ancestor.h

3b29e6, 9 lines

```
int kth_ancestor(int node,int k){
    if(dep[node] < k)return -1;
    for(int i = 0;i < LOG; ++i){
        if(k & (1<<i)){
            node = up[node][i];
        }
    }
    return node;
}
```

tree-hashing.h

Description: very acurate rooted tree hashing(I don't know how), don't forget to check tree invariant such as tree size and numbers of centroid

ce0886, 18 lines

```
map<vector<int>, int> hasher;

int hashify(vector<int> x) {
    sort(x.begin(), x.end());
    if(!hasher[x]) {
        hasher[x] = hasher.size();
    }
    return hasher[x];
}

int _hash(int v, int e, vector<vector<int>>&adj) { // get a "
     hash" of v's subtree, e is parent vertex
    vector<int> children;
    for(int u: adj[v]) {
        if( u == e )continue;
        children.push_back(_hash(u, v, adj));
    }
    return hashify(children);
}
```

prim-mst.h

Description: unlike kruskal, prim's algorithm assume all vertices belong in same graph

a70ceb, 23 lines

```
vector<pair<int,int>>adj[N+1];
bool taken[N+1];
priority_queue<pair<int,int>>PQ;

void process(int v){
    taken[v]=1;
    for(auto&p:adj[v]){
        int b=p.second, w=p.first;
        if(!taken[b])PQ.push({-w,-b});
    }
}

ll mst(){
    process(1);
    ll mst_cost=0;
    while(!PQ.empty()){
        pair<int,int> front=PQ.top(); PQ.pop();
        int u=-front.second, w=-front.first;
        if(!taken[u])
            mst_cost+=w, process(u);
    }
    return mst_cost;
}
```

superset-and-subset-iteration.h

Description: both work in O(3**n)

204064, 12 lines

```
for (int s = 0; s < (1<<n); s++) {
    for (int t = s; t < (1<<n); t = (t+1)|s) {
        // t is a superset of s
```

```
    }
}

for (int s = 0; s < (1<<n); s++) {
    for (int t = s; ; t = (t-1)&s) {
        // t is a subset of s
        if (t == 0) break;
    }
}
```

gp-hash-table.h

<ext/pb_ds/assoc_container.hpp> 3b295b, 7 lines

```
using namespace __gnu_pbds;

const int RANDOM = chrono::high_resolution_clock::now().
    time_since_epoch().count();
struct chash {
    int operator()(int x) const { return x ^ RANDOM; }
};
gp_hash_table<int, int, chash> table;
```

segment-tree-lazy-propagation.h

Description: monoid combiner function

740cd4, 59 lines

```
vector<long long> tree,lazy;
void update(int node,int n_l,int n_r,int q_l,int q_r,long long
    value){
    // Push: Process and clear pending lazy tags down to
        children
    if(lazy[node]!=0){
        tree[node]+=(long long)(n_r-n_l+1)*lazy[node];
        if(n_l!=n_r){
            lazy[2*node]+=lazy[node];
            lazy[2*node+1]+=lazy[node];
        }
        lazy[node] = 0;
    }

    if(n_r<q_l || q_r<n_l)return;

    // update tree node and set lazy tags for children
    if(q_l<=n_l && n_r<=q_r){
        tree[node]+=(long long)(n_r-n_l+1)*value;
        if(n_l!=n_r){
            lazy[2*node]+=value;
            lazy[2*node+1]+=value;
        }
        return;
    }

    int mid = (n_r+n_l)/2;
    update(2*node,n_l,mid,q_l,q_r,value);
    update(2*node+1,mid+1,n_r,q_l,q_r,value);

    tree[node] = tree[2*node] + tree[2*node+1];
}

long long f(int node,int n_l,int n_r,int q_l,int q_r){
    // Push: Process and clear pending lazy tags down to
        children
    if(lazy[node]!=0){
        tree[node]+=(long long)(n_r-n_l+1)*lazy[node];
        if(n_l!=n_r){
            lazy[2*node] += lazy[node];
            lazy[2*node+1] += lazy[node];
        }
        lazy[node] = 0;
    }

    if(n_r<q_l || q_r<n_l)return 0;

    if(q_l<=n_l && n_r<=q_r)return tree[node];

    int mid = (n_l+n_r)/2;
    return f(2*node,n_l,mid,q_l,q_r) + f(2*node+1,mid+1,n_r,q_l
        ,q_r);
}

int build_tree(vi &a,int n){
    tree.clear(); lazy.clear();
    int m=n;
    while(__builtin_popcount(m)!=1)++m;
    tree.resize(2*m+10,0); lazy.resize(2*m+10,0);
    for(int i=0;i<n;++i)tree[i+m]=a[i];
    for(int i=m-1;i>=1;--i)tree[i]=tree[2*i]+tree[2*i+1];
    return m;
}
```

tarjan-scc.h

Description: we can use this to solve 2-SAT

3e5381, 35 lines

```
//Tarjan's algorithm
vi adj[N+1];
int id[N+1], low[N+1];
bool vst[N+1], in_stack[N+1];

stack<int>st;
vector<vi>scc;
vi comp;
int tin=0;
void dfs(int a){
    st.push(a);
    vst[a]=in_stack[a]=1;
    id[a]=low[a]=tin++;

    for(auto&b:adj[a]){
        if(!vst[b]){
            dfs(b);
            low[a]=min(low[a],low[b]); //Backtrack low-link
                value
        } else if(in_stack[b]){
            //Backtrack low-link value
            low[a]=min(low[a],low[b]);
        }
    }
    if(id[a] == low[a]){ //pop scc
        while(!st.empty()){
            int v=st.top();
            comp.pb(v), st.pop();
            in_stack[v]=0;
            if(v==a)break; //stop when scc is found
        }
    }
    if(!comp.empty()){
        scc.pb(comp), comp.clear();
    }
}
```

bridge-finding.h

Description: this not work for multigraph, I'm too lazy too fix for now

747a19, 29 lines

```
/*
    dfs-tree bridge finding algorithm
    let lvl[root] = 1 (root = 1)
*/
int root = 1;
vector<pair<int,int>> bridge;
int dp[N+1], lvl[N+1];

void dfs_tree(int a, int e){
    if(a == root)lvl[a] = 1;
    dp[a] = 0;
    for(auto &b:adj[a]){
        if(b == e)continue;

        if(lvl[b] == 0){
            lvl[b] = lvl[a] + 1;
            dfs_tree(b, a);
            dp[a] += dp[b];
        } else if(lvl[b] < lvl[a]){
            dp[a]++;
        } else if(lvl[b] > lvl[a]){
            dp[a]--;
        }
    }

    if(lvl[a] > 1 and dp[a] == 0){
        bridge.pb({min(a,e), max(a,e)});
    }
}
```

matrix.h

73a80f, 33 lines

```
struct Matrix {
    vector<vector<long long>> a;
    int rows, cols;

    Matrix(int r, int c, long long val) : rows(r), cols(c), a(r
        , vector<long long>(c, val)) {}

    Matrix operator *(const Matrix& other) {
        Matrix product(rows, other.cols, 0);

        for (int i = 0; i < rows; ++i) {
            for (int j = 0; j < other.cols; ++j) {
                for (int k = 0; k < cols; ++k) {
                    product.a[i][j] += a[i][k] * other.a[k][j]
                        % MOD;
                    product.a[i][j] %= MOD;
                }
            }
        }
        return product;
    }
};

Matrix expo_power(Matrix a, long long n, int size){
    Matrix product(size,size,0);
    for(int i=0;i<size;++i)product.a[i][i]=1;
    while(n>0){
        if(n&1){
            product = product * a;
        }
        a = a * a;
        n>>=1;
    }
    return product;
}
```

gauss-jordan-elimination.h

d847fe, 48 lines

```
const double EPS = 1e-9;
const int INF = 2; // it doesn't actually have to be infinity
    or a big number

int gauss (vector < vector<double> > a, vector<double> & ans) {
    int n = (int) a.size();
```

```
    int m = (int) a[0].size() - 1;

    vector<int> where (m, -1);
    for (int col=0, row=0; col<m && row<n; ++col) {
        int sel = row;
        for (int i=row; i<n; ++i)
            if (abs (a[i][col]) > abs (a[sel][col]))
                sel = i;

        if (abs (a[sel][col]) < EPS)
            continue;

        for (int i=col; i<=m; ++i)
            swap (a[sel][i], a[row][i]);
        where[col] = row;

        for (int i=0; i<n; ++i)
            if (i != row) {
                double c = a[i][col] / a[row][col];
                for (int j=col; j<=m; ++j)
                    a[i][j] -= a[row][j] * c;
            }
        ++row;
    }

    ans.assign (m, 0);
    for (int i=0; i<m; ++i)
        if (where[i] != -1)
            ans[i] = a[where[i]][m] / a[where[i]][i];

    for (int i=0; i<n; ++i) {
        double sum = 0;
        for (int j=0; j<m; ++j)
            sum += ans[j] * a[i][j];
        if (abs (sum - a[i][m]) > EPS)
            return 0; // contradiction means no solution
    }

    for (int i=0; i<m; ++i)
        if (where[i] == -1)
            return INF; //infinite solution
    return 1; //unique solution
}
```

2.4 General Data Structures & Algorithms

aho-corasick-automaton.h

Description: trie structure can be infer from this

a41621, 53 lines

```
const int K = 26;

struct Vertex {
    int next[K];
    bool output = false;
    int p = -1;
    char pch;
    int link = -1;
    int go[K];

    Vertex(int p=-1, char ch='$') : p(p), pch(ch) {
        fill(begin(next), end(next), -1);
        fill(begin(go), end(go), -1);
    }
};

vector<Vertex> t(1);

void add_string(string const& s) {
    int v = 0;
    for (char ch : s) {
        int c = ch - 'a';
        if (t[v].next[c] == -1) {
            t[v].next[c] = t.size();
            t.emplace_back(v, ch);
        }
        v = t[v].next[c];
    }
    t[v].output = true;
}

int go(int v, char ch);

int get_link(int v) {
    if (t[v].link == -1) {
        if (v == 0 || t[v].p == 0)
            t[v].link = 0;
        else
            t[v].link = go(get_link(t[v].p), t[v].pch);
    }
    return t[v].link;
}

int go(int v, char ch) {
    int c = ch - 'a';
    if (t[v].go[c] == -1) {
        if (t[v].next[c] != -1)
            t[v].go[c] = t[v].next[c];
        else
            t[v].go[c] = v == 0 ? 0 : go(get_link(v), ch);
    }
    return t[v].go[c];
}
```

string-hashing.h

ba27df, 17 lines

```
typedef __int128 HASH;
HASH p[NN], h[NN];
constexpr HASH M = 1000000000000000003;
mt19937 rng(chrono::steady_clock::now().time_since_epoch().
    count());
#define uid(a, b) uniform_int_distribution<long long>(a, b)(rng
    )

void compute_hash(string const& s){
    p[0] = 1, p[1] = uid(256, M-1);
    for(ll i=0;i<(int)s.length();++i){
        p[i+1] = p[i]*p[1]%M;
        h[i+1] = (h[i]*p[1] + s[i])%M;
    }
}

HASH sub_hash(ll l, ll r){ //range half-open [l, r)
    return (h[r] - p[r-l]*h[l]%M + M)%M;
}
```

suffix-automaton.h

Description: I vaguely remember how to use this

fe30a6, 52 lines

```
struct state {
    int len, link;
    map<char, int> next;
    map<char, int> go;
};

const int MAXLEN = 100001; //this one is 10^5 becareful
state st[MAXLEN * 2];
int sz, last;

void sa_init() {
    st[0].len = 0;
    st[0].link = -1;
    sz++;
    last = 0;
}

void sa_extend(char c) {
    int cur = sz++;
    st[cur].len = st[last].len + 1;
    int p = last;
    while (p != -1 && !st[p].next.count(c)) {
        st[p].next[c] = cur;
        p = st[p].link;
    }
    if (p == -1) {
        st[cur].link = 0;
    } else {
        int q = st[p].next[c];
        if (st[p].len + 1 == st[q].len) {
            st[cur].link = q;
        } else {
            int clone = sz++;
            st[clone].len = st[p].len + 1;
            st[clone].next = st[q].next;
            st[clone].link = st[q].link;
            while (p != -1 && st[p].next[c] == q) {
                st[p].next[c] = clone;
                p = st[p].link;
            }
            st[q].link = st[cur].link = clone;
        }
    }
    last = cur;
}

int sa_go(int v, char c) {
    if (st[v].go.count(c)) return st[v].go[c];
    if (st[v].next.count(c)) return st[v].go[c] = st[v].next[c
        ];
    if (st[v].link == -1) return st[v].go[c] = 0; // fallback
        to root
    return st[v].go[c] = sa_go(st[v].link, c);
}
```

kmp.h

Description: we only need prefix function, pi[i] is the length of the longest proper prefix of the substring s[0...i] which is also a suffix of this substring, suffix link can be infer from this.

4dfee5, 37 lines

```
int b[N];
int cnt = 0;

void knp_proc(string t,string p){
    int i=0,j=-1;b[0] = -1;
    while(i<(int)p.length()){
        while(j>=0 && p[i]!=p[j]) j =b[j];
        ++i;++j;
        b[i] = j;
    }
}

void knp_search(string t,string p){//count number of occurence
    of p in t
    int i=0,j=0;
    while(i<(int)t.length()){
        while(j>=0 && t[i] != p[j]) j = b[j];
        ++i;++j;
```

```
        if(j==(int)p.length()){
            ++cnt;
            j = b[j];
        }
    }
}

vector<int> prefix_function(string s) {
    int n = (int)s.length();
    vector<int> pi(n);
    for (int i = 1; i < n; i++) {
        int j = pi[i-1];
        while (j > 0 && s[i] != s[j])
            j = pi[j-1];
        if (s[i] == s[j])
            j++;
        pi[i] = j;
    }
    return pi;
}
```

z-function.h

Description: Description: z[i] is the length of the longest common prefix between s and the suffix of s starting at i.

9a2512, 18 lines

```
vector<int> z_function(string s) {
    int n = s.size();
    vector<int> z(n);
    int l = 0, r = 0;
    for(int i = 1; i < n; i++) {
        if(i < r) {
            z[i] = min(r - i, z[i - l]);
        }
        while(i + z[i] < n && s[z[i]] == s[i + z[i]]) {
            z[i]++;
        }
        if(i + z[i] > r) {
            l = i;
            r = i + z[i];
        }
    }
    return z;
}
```

manacher.h

3069fe, 38 lines

```
//sub-palindrome queries
vector<int> manacher_odd(string s) {
    int n = s.size();
    s = "$" + s + "^";
    vector<int> p(n + 2);
    int l = 1, r = 1;
    for(int i = 1; i <= n; i++) {
        p[i] = max(0, min(r - i, p[l + (r - i)]));
        while(s[i - p[i]] == s[i + p[i]]) {
            p[i]++;
        }
        if(i + p[i] > r) {
            l = i - p[i], r = i + p[i];
        }
    }
    return vector<int>(begin(p) + 1, end(p) - 1);
}

vector<int> manacher(string s) {
    string t;
    for(auto c: s) {
        t += string("#") + c;
    }
    auto res = manacher_odd(t + "#");
    return vector<int>(begin(res) + 1, end(res) - 1);
}

//beware the range is inclusive and therefore not half-open -> [l, r]
bool is_palindrome(int l, int r, vector<int> &v){
    if(v[l +r] < (r - l + 1))return false;
    return true;
}

vector<int> build_manacher(string s){ //0 base index string
    auto v = manacher(s);
    for(auto&x:v)--x;
    return v;
}
```

suffix-array.h

Description: I don't fucking know how to use this

4ca004, 53 lines

```
vector<int> sort_cyclic_shifts(string const& s) {
    int n = s.size();
    const int alphabet = 256;

    vector<int> p(n), c(n), cnt(max(alphabet, n), 0);
    for (int i = 0; i < n; i++)
        cnt[s[i]]++;
    for (int i = 1; i < alphabet; i++)
        cnt[i] += cnt[i-1];
    for (int i = 0; i < n; i++)
        p[--cnt[s[i]]] = i;
    c[p[0]] = 0;
    int classes = 1;
    for (int i = 1; i < n; i++) {
        if (s[p[i]] != s[p[i-1]])
            classes++;
        c[p[i]] = classes - 1;
    }

    vector<int> pn(n), cn(n);
    for (int h = 0; (1 << h) < n; ++h) {
        for (int i = 0; i < n; i++) {
            pn[i] = p[i] - (1 << h);
            if (pn[i] < 0)
                pn[i] += n;
        }
        fill(cnt.begin(), cnt.begin() + classes, 0);
        for (int i = 0; i < n; i++)
            cnt[c[pn[i]]]++;
        for (int i = 1; i < classes; i++)
            cnt[i] += cnt[i-1];
        for (int i = n-1; i >= 0; i--)
            p[--cnt[c[pn[i]]]] = pn[i];
        cn[p[0]] = 0;
        classes = 1;
        for (int i = 1; i < n; i++) {
            pair<int, int> cur = {c[p[i]], c[(p[i] + (1 << h)) % n]};
            pair<int, int> prev = {c[p[i-1]], c[(p[i-1] + (1 << h)) % n]};
            if (cur != prev)
                ++classes;
            cn[p[i]] = classes - 1;
        }
        c.swap(cn);
    }
    return p;
}

vector<int> suffix_array_construction(string s) {
    s += "$";
    vector<int> sorted_shifts = sort_cyclic_shifts(s);
    sorted_shifts.erase(sorted_shifts.begin());
    return sorted_shifts;
}
```

lyndon-factorization.h

Description: hope that we may never need this

a3cc8c, 40 lines

```
vector<string> duval(string const& s) {
    int n = s.size();
    int i = 0;
    vector<string> factorization;
    while (i < n) {
        int j = i + 1, k = i;
        while (j < n && s[k] <= s[j]) {
            if (s[k] < s[j])
                k = i;
            else
                k++;
            j++;
        }
        while (i <= k) {
            factorization.push_back(s.substr(i, j - k));
            i += j - k;
        }
    }
    return factorization;
}

string min_cyclic_string(string s) {
    s += s;
    int n = s.size();
    int i = 0, ans = 0;
    while (i < n / 2) {
        ans = i;
        int j = i + 1, k = i;
        while (j < n && s[k] <= s[j]) {
            if (s[k] < s[j])
                k = i;
            else
                k++;
            j++;
        }
        while (i <= k)
            i += j - k;
    }
    return s.substr(ans, n / 2);
}
```

2.5 Geometry

geometry.h

963fd3, 167 lines

```
//Geometry
#pragma GCC target("avx2")
#pragma GCC optimize("O3")
#pragma GCC optimize("unroll-loops")

typedef long long int ll;
typedef long double ld;
typedef complex<ld> pt;
struct line {
  pt P, D; bool S = false;
  line(pt p, pt q, bool b = false) : P(p), D(q - p), S(b) {}
  line(pt p, ld th) : P(p), D(polar((ld)1, th)) {}
};
struct circ { pt C; ld R; };
```

```
#define X real()
#define Y imag()
#define CRS(a, b) (conj(a) * (b)).Y //scalar cross product
#define DOT(a, b) (conj(a) * (b)).X //dot product
#define U(p) ((p) / abs(p)) //unit vector in direction of p (
    don't use if Z(p) == true)
#define Z(x) (abs(x) < EPS)
#define A(a) (a).begin(), (a).end() //shortens sort(),
    upper_bound(), etc. for vectors

//constants (INF and EPS may need to be modified)
ld PI = acosl(-1), INF = 1e20, EPS = 1e-12;
pt I = {0, 1};

//true if d1 and d2 parallel (zero vectors considered parallel
    to everything)
bool parallel(pt d1, pt d2) { return Z(d1) || Z(d2) || Z(CRS(U(
    d1), U(d2))); }

//"above" here means if l & p are rotated such that l.D points
    in the +x direction, then p is above l. Returns arbitrary
    boolean if p is on l
bool above_line(pt p, line l) { return CRS(p - l.P, l.D) > 0; }

//true if p is on line l
bool on_line(pt p, line l) { return parallel(l.P - p, l.D) &&
    (!l.S || DOT(l.P - p, l.P + l.D - p) <= EPS); }

//returns 0 for no intersection, 2 for infinite intersections,
    1 otherwise. p holds intersection pt
ll intsct(line l1, line l2, pt& p) {
  if(parallel(l1.D, l2.D)) //note that two parallel segments
      sharing one endpoint are considered to have infinite
      intersections here
    return 2 * (on_line(l1.P, l2) || on_line(l1.P + l1.D, l2)
        || on_line(l2.P, l1) || on_line(l2.P + l2.D, l1));
  pt q = l1.P + l1.D * CRS(l2.D, l2.P - l1.P) / CRS(l2.D, l1.D)
      ;
  if(on_line(q, l1) && on_line(q, l2)) { p = q; return 1; }
  return 0;
}

//closest pt on l to p
pt cl_pt_on_l(pt p, line l) {
  pt q = l.P + DOT(U(l.D), p - l.P) * U(l.D);
  if(on_line(q, l)) return q;
  return abs(p - l.P) < abs(p - l.P - l.D) ? l.P : l.P + l.D;
}

//distance from p to l
ld dist_to(pt p, line l) { return abs(p - cl_pt_on_l(p, l)); }

//p reflected over l
pt refl_pt(pt p, line l) { return conj((p - l.P) / U(l.D)) * U(
    l.D) + l.P; }

//ray r reflected off l (if no intersection, returns original
    ray)
line reflect_line(line r, line l) {
  pt p; if(intsct(r, l, p) - 1) return r;
  return line(p, p + INF * (p - refl_pt(r.P, l)), 1);
}

//altitude from p to l
line alt(pt p, line l) { l.S = 0; return line(p, cl_pt_on_l(p,
    l)); }

//angle bisector of angle abc
line ang_bis(pt a, pt b, pt c) { return line(b, b + INF * (U(a
    - b) + U(c - b)), 1); }

//perpendicular bisector of l (assumes l.S == 1)
line perp_bis(line l) { return line(l.P + l.D / (ld)2, arg(l.D
    * I)); }

//orthocenter of triangle abc
pt orthocent(pt a, pt b, pt c) { pt p; intsct(alt(a, line(b, c)
    ), alt(b, line(a, c)), p); return p; }

//incircle of triangle abc
circ incirc(pt a, pt b, pt c) {
  pt cent; intsct(ang_bis(a, b, c), ang_bis(b, a, c), cent);
  return {cent, dist_to(cent, line(a, b))};
}

//circumcircle of triangle abc
circ circumcirc(pt a, pt b, pt c) {
  pt cent; intsct(perp_bis(line(a, b, 1)), perp_bis(line(a, c,
      1)), cent);
  return {cent, abs(cent - a)};
}

//is pt p inside the (not necessarily convex) polygon given by
    poly
bool in_poly(pt p, vector<pt>& poly) {
  line l = line(p, {INF, INF * PI}, 1);
  bool ans = false;
  pt lst = poly.back(), tmp;
  for(pt q : poly) {
    line s = line(q, lst, 1); lst = q;
    if(on_line(p, s)) return false; //change if border included
    else if(intsct(l, s, tmp)) ans = !ans;
  }
  return ans;
}

//area of polygon, vertices in order (cw or ccw)
ld area(vector<pt>& poly) {
  ld ans = 0;
  pt lst = poly.back();
  for(pt p : poly) ans += CRS(lst, p), lst = p;
  return abs(ans / 2);
}

//perimeter of polygon, vertices in order (cw or ccw)
ld perim(vector<pt>& poly) {
  ld ans = 0;
  pt lst = poly.back();
  for(pt p : poly) ans += abs(lst - p), lst = p;
  return ans;
}

//centroid of polygon, vertices in order (cw or ccw)
pt centroid(vector<pt>& poly) {
  ld area = 0;
  pt lst = poly.back(), ans = {0, 0};
  for(pt p : poly) {
    area += CRS(lst, p);
    ans += CRS(lst, p) * (lst + p) / (ld)3;
    lst = p;
  }
  return ans / area;
}

//invert a point over a circle (doesn't work for center of
    circle)
pt circInv(pt p, circ c) {
    return c.R * c.R / conj(p - c.C) + c.C;
}

//vector of intersection pts of two circs (up to 2) (if circles
     same, returns empty vector)
vector<pt> intsctCC(circ c1, circ c2) {
  if(c1.R < c2.R) swap(c1, c2);
  pt d = c2.C - c1.C;
  if(Z(abs(d) - c1.R - c2.R)) return {c1.C + polar(c1.R, arg(c2
      .C - c1.C))};
  if(!Z(d) && Z(abs(d) - c1.R + c2.R)) return {c1.C + c1.R * U(
      d)};
  if(abs(abs(d) - c1.R) >= c2.R - EPS) return {};
  ld th = acosl((c1.R * c1.R + norm(d) - c2.R * c2.R) / (2 * c1
      .R * abs(d)));
  return {c1.C + polar(c1.R, arg(d) + th), c1.C + polar(c1.R,
      arg(d) - th)};
}

//vector of intersection pts of a line and a circ (up to 2)
vector<pt> intsctCL(circ c, line l) {
  vector<pt> v, ans;
  if(parallel(l.D, c.C - l.P)) v = {c.C + c.R * U(l.D), c.C - c
      .R * U(l.D)};
  else v = intsctCC(c, circ{refl_pt(c.C, l), c.R});
  for(pt p : v) if(on_line(p, l)) ans.push_back(p);
  return ans;
}

//external tangents of two circles (negate c2.R for internal
    tangents)
vector<line> circTangents(circ c1, circ c2) {
  pt d = c2.C - c1.C;
  ld dr = c1.R - c2.R, d2 = norm(d), h2 = d2 - dr * dr;
  if(Z(d2) || h2 < 0) return {};
  vector<line> ans;
  for(ld sg : {-1, 1}) {
    pt u = (d * dr + d * I * sqrt(h2) * sg) / d2;
    ans.push_back(line(c1.C + u * c1.R, c2.C + u * c2.R, 1));
  }
  if(Z(h2)) ans.pop_back();
  return ans;
}
```

closest-pair-of-points.h

85eeeb, 67 lines

```
struct pt {
    int x, y, id;
};

struct cmp_x {
    bool operator()(const pt & a, const pt & b) const {
        return a.x < b.x || (a.x == b.x && a.y < b.y);
    }
};

struct cmp_y {
    bool operator()(const pt & a, const pt & b) const {
        return a.y < b.y;
    }
};

int n;
vector<pt> a;

double mindist;
pair<int, int> best_pair;

void upd_ans(const pt & a, const pt & b) {
```

```
    double dist = sqrt((a.x - b.x)*(a.x - b.x) + (a.y - b.y)*(a
        .y - b.y));
    if (dist < mindist) {
        mindist = dist;
        best_pair = {a.id, b.id};
    }
}

vector<pt> t;

void rec(int l, int r) {
    if (r - l <= 3) {
        for (int i = l; i < r; ++i) {
            for (int j = i + 1; j < r; ++j) {
                upd_ans(a[i], a[j]);
            }
        }
        sort(a.begin() + l, a.begin() + r, cmp_y());
        return;
    }

    int m = (l + r) >> 1;
    int midx = a[m].x;
    rec(l, m);
    rec(m, r);

    merge(a.begin() + l, a.begin() + m, a.begin() + m, a.begin
        () + r, t.begin(), cmp_y());
    copy(t.begin(), t.begin() + r - l, a.begin() + l);

    int tsz = 0;
    for (int i = l; i < r; ++i) {
        if (abs(a[i].x - midx) < mindist) {
            for (int j = tsz - 1; j >= 0 && a[i].y - t[j].y <
                 mindist; --j)
                upd_ans(a[i], t[j]);
            t[tsz++] = a[i];
        }
    }
}

void call_rec(){
    t.resize(n);
    sort(a.begin(), a.end(), cmp_x());
    mindist = 1E20;
    rec(0, n);
}
```

2.6 Advanced Algorithms

fft.h

Description: from cp algorithm the inplace-computation fft part

cbf4b6, 62 lines

```
using cd = complex<double>;
const double PI = acos(-1);

int reverse(int num, int lg_n) {
    int res = 0;
    for (int i = 0; i < lg_n; i++) {
        if (num & (1 << i))
            res |= 1 << (lg_n - 1 - i);
    }
    return res;
}

void fft(vector<cd> & a, bool invert) {
    int n = a.size();
    int lg_n = 0;
    while ((1 << lg_n) < n)
        lg_n++;

    for (int i = 0; i < n; i++) {
        if (i < reverse(i, lg_n))
            swap(a[i], a[reverse(i, lg_n)]);
    }

    for (int len = 2; len <= n; len <<= 1) {
        double ang = 2 * PI / len * (invert ? -1 : 1);
        cd wlen(cos(ang), sin(ang));
        for (int i = 0; i < n; i += len) {
            cd w(1);
            for (int j = 0; j < len / 2; j++) {
                cd u = a[i+j], v = a[i+j+len/2] * w;
                a[i+j] = u + v;
                a[i+j+len/2] = u - v;
                w *= wlen;
            }
        }
    }

    if (invert) {
        for (cd & x : a)
            x /= n;
    }
}

vector<int> multiply(vector<int> const& a, vector<int> const& b
    ) {
    vector<cd> fa(a.begin(), a.end()), fb(b.begin(), b.end());
    int n = 1;
    while (n < (int)a.size() + (int)b.size())
        n <<= 1;
    fa.resize(n);
    fb.resize(n);

    fft(fa, false);
    fft(fb, false);
    for (int i = 0; i < n; i++)
        fa[i] *= fb[i];
    fft(fa, true);

    vector<int> result(n);
    for (int i = 0; i < n; i++)
        result[i] = round(fa[i].real());
    return result;
}
```

persistent-segment-tree.h

Description: Warning!This implementation is not garbage collected, so the tree nodes aren't deleted even if the instance of the segment tree is taken out of scope.

c4a26b, 37 lines

```
struct Node {
    ll val;
  Node *l, *r;

  Node(ll x) : val(x), l(nullptr), r(nullptr) {}
  Node(Node *ll, Node *rr) {
    l = ll, r = rr;
    val = 0;
    if (l) val += l->val;
    if (r) val += r->val;
  }
  Node(Node *cp) : val(cp->val), l(cp->l), r(cp->r) {}
};

int n, cnt = 1;
ll a[200001];
Node *roots[200001];

Node *build(int l = 1, int r = n) {
  if (l == r) return new Node(a[l]);
  int mid = (l + r) / 2;
  return new Node(build(l, mid), build(mid + 1, r));
}

Node *update(Node *node, int val, int pos, int l = 1, int r = n
    ) {
  if (l == r) return new Node(val);
  int mid = (l + r) / 2;
  if (pos > mid) return new Node(node->l, update(node->r, val,
      pos, mid + 1, r));
  else return new Node(update(node->l, val, pos, l, mid), node
      ->r);
}

ll query(Node *node, int a, int b, int l = 1, int r = n) {
  if (l > b || r < a) return 0;
  if (l >= a && r <= b) return node->val;
  int mid = (l + r) / 2;
  return query(node->l, a, b, l, mid) + query(node->r, a, b,
      mid + 1, r);
}
```

mo2D.h

9df2fb, 49 lines

```
/* always use zero based indexing*/
/* there N/B blocks so left move ~ N/B*B + Q*B, right move ~ N/
    B*B + (N/B)*B */
/* so O(N^2/B + N + QB) ~~ O((N+Q)sqrt(N)) when B = sqrt(N) */
/* when Q~N it's ~ O(N^3/2) */
void remove(int idx);  // TODO: remove value at idx from data
    structure
void add(int idx);     // TODO: add value at idx from data
    structure
int get_answer();  // TODO: extract the current answer of the
    data structure

int block_size;

struct Query {
    int l, r, idx;
    bool operator<(Query other) const
    {
        return make_pair(l / block_size, r) <
               make_pair(other.l / block_size, other.r);
    }
};

vector<int> mo_s_algorithm(vector<Query>& queries) {
    vector<int> answers(queries.size());
    sort(queries.begin(), queries.end());

    // TODO: initialize data structure

    int cur_l = 0;
    int cur_r = -1;
    // invariant: data structure will always reflect the range
        [cur_l, cur_r]
    for (Query q : queries) {
        while (cur_l > q.l) {
            cur_l--;
            add(cur_l);
        }
        while (cur_r < q.r) {
            cur_r++;
            add(cur_r);
        }
```

```
        while (cur_l < q.l) {
            remove(cur_l);
            cur_l++;
        }
        while (cur_r > q.r) {
            remove(cur_r);
            cur_r--;
        }
        answers[q.idx] = get_answer();
    }
    return answers;
}
```

mo3D.h

dc4a5e, 62 lines

```
/* always use zero based indexing*/
/* ~ O(N^(5/3)) */
int block_size;

struct Query {
    int l, r, t, idx;
    bool operator<(Query other) const
    {
        return make_tuple(l / block_size, r / block_size, t) <
               make_tuple(other.l / block_size, other.r /
                    block_size,other.t);
    }
};

int ans[N];
void mo_s_algorithm(vector<Query>& queries) {
    sort(queries.begin(), queries.end());
    // TODO: initialize data structure

    int cur_l = 0;
    int cur_r = -1;
    int cur_t = -1;
    // invariant: data structure will always reflect the range
         [cur_l, cur_r]
    for (Query q : queries) {
        while (cur_l > q.l) {
            cur_l--;
            //add(cur_l)
        }
        while (cur_r < q.r) {
            cur_r++;
            //add(cur_r)
        }
        while (cur_l < q.l) {
            //remove(cur_l);
            cur_l++;
        }
        while (cur_r > q.r) {
            //remove(cur_r);
            cur_r--;
        }

        while(cur_t < q.t){
            cur_t++;
            //update
        }
        while(cur_t > q.t){
            //update rollback
            cur_t--;
        }
        ans[q.idx] = get_answer(); // declare array for this
    }
}
```

```
void getBlockSize(int n, int q){
    if (q >= n) {
        // most common: Q ~ N
        block_size = max(1, (int)pow(n, 2.0/3.0));
    } else {
        // general cube-root rule
        block_size = max(1, (int)pow((long double)n*n / q,
             1.0/3.0));
    }
    // then sort your queries by (L/B, R/B, T) or via Hilbert
}
```

cht.h

87ad47, 28 lines

```
struct line {
    long long m, c;
    long long eval(long long x) { return m * x + c; }
    long double intersectX(line l) { return (long double) (c -
         l.c) / (l.m - m); }
};

deque<line> dq;
dq.push_front({0, 0});//cant be put in global, remove this to
     local function
//if query ask for minimum remove this line after 1st insertion

//TODO NOTE***: maximum and minimum value exist in bot left
     most and rightmost of convex hull so do search on both l
     to r and r to l

//constructing hull from l to r, maintain correct hull at
     rightmost
/*  ***inserting line (maximum hull)
    line cur = line{...some m, ...some c}
    while(dq.size()>=2&&cur.intersectX(dq.back())
       <=cur.intersectX(dq[dq.size()-2]))dq.pop_back();
    dq.pb(cur);
*/

//constructing hull from r to l, maintain correct hull at
     leftmost
/*  inserting line (maximum hull)
    line cur = line{...some m, ...some c}
    while(dq.size()>=2&&cur.intersectX(dq[0])>=cur.intersectX(
         dq[1])){
        dq.pop_front();
    }
    dq.push_front(cur);
*/
```

dinic.h

ce6220, 82 lines

```
//COPY FROM BENQ GITHUB I dont fucking know how this work
//General graphs: O(V^2E);
//Bipartite matching(unit caps): O(E sqrt2(V))
struct Dinic {
    using F = long long; // flow type
    struct Edge { int to; F flow, cap; };
    int N;
    vector<Edge> edges;
    vector<vector<int>> adj;
    vector<int> level;
    vector<vector<int>::iterator> it;

    // Initialize Dinic for a graph with _N vertices (0-indexed
         or 1-indexed)
    void init(int _N) {
        N = _N;
        adj.assign(N, {});
        edges.clear();
        level.resize(N);
        it.resize(N);
    }

    // Add edge u -> v with capacity 'cap', and reverse edge v
         -> u with capacity 'rev_cap'
    void addEdge(int u, int v, F cap, F rev_cap = 0) {
        // forward edge
        adj[u].push_back((int)edges.size());
        edges.push_back({v, 0, cap});
        // reverse edge
        adj[v].push_back((int)edges.size());
        edges.push_back({u, 0, rev_cap});
    }

    // Build level graph via BFS
    bool bfs(int s, int t) {
        fill(level.begin(), level.end(), -1);
        for (int i = 0; i < N; i++) it[i] = adj[i].begin();
        queue<int> q;
        level[s] = 0;
        q.push(s);
        while (!q.empty()) {
            int u = q.front(); q.pop();
            for (int idx : adj[u]) {
                const Edge &e = edges[idx];
                if (level[e.to] < 0 && e.flow < e.cap) {
                    level[e.to] = level[u] + 1;
                    q.push(e.to);
                }
            }
        }
        return level[t] >= 0;
    }

    // DFS to send flow
    F dfs(int u, int t, F pushed) {
        if (u == t || pushed == 0) return pushed;
        for (; it[u] != adj[u].end(); ++it[u]) {
            int idx = *it[u];
            Edge &e = edges[idx];
            if (level[e.to] != level[u] + 1 || e.flow == e.cap)
                 continue;
            F tr = dfs(e.to, t, min(pushed, e.cap - e.flow));
            if (tr > 0) {
                e.flow += tr;
                edges[idx ^ 1].flow -= tr; // reverse edge
                return tr;
            }
        }
        return 0;
    }

    // Compute max flow from s to t
    F maxFlow(int s, int t) {
        F flow = 0;
        while (bfs(s, t)) {
            while (F pushed = dfs(s, t, numeric_limits<F>::max
                 ())) {
                flow += pushed;
            }
        }
        return flow;
    }

    // After maxFlow, vertices reachable from s in residual
         graph have level >= 0.
    // Edges from reachable u to unreachable v form a min-cut.
```

```
};
```

articulation-points.h

Description: IS-CUTPOINT(v) in this code is not implemented, also I never use this and don't know how does this works

23f413, 37 lines

```
int n; // number of nodes
vector<vector<int>> adj; // adjacency list of graph

vector<bool> visited;
vector<int> tin, low;
int timer;

void dfs(int v, int p = -1) {
    visited[v] = true;
    tin[v] = low[v] = timer++;
    int children=0;
    for (int to : adj[v]) {
        if (to == p) continue;
        if (visited[to]) {
            low[v] = min(low[v], tin[to]);
        } else {
            dfs(to, v);
            low[v] = min(low[v], low[to]);
            if (low[to] >= tin[v] && p!=-1)
                IS_CUTPOINT(v);
            ++children;
        }
    }
    if(p == -1 && children > 1)
        IS_CUTPOINT(v);
}

void find_cutpoints() {
    timer = 0;
    visited.assign(n, false);
    tin.assign(n, -1);
    low.assign(n, -1);
    for (int i = 0; i < n; ++i) {
        if (!visited[i])
            dfs (i);
    }
}
```

hld.h

Description: simple hld that I copy from cp algorithm

2ddd4c, 67 lines

```
vector<int> parent, depth, heavy, head, pos;
int cur_pos;

int dfs(int v, vector<vector<int>> const& adj) {
    int size = 1;
    int max_c_size = 0;
    for (int c : adj[v]) {
        if (c != parent[v]) {
            parent[c] = v, depth[c] = depth[v] + 1;
            int c_size = dfs(c, adj);
            size += c_size;
            if (c_size > max_c_size)
                max_c_size = c_size, heavy[v] = c;
        }
    }
    return size;
}

void decompose(int v, int h, vector<vector<int>> const& adj) {
    head[v] = h, pos[v] = cur_pos++;
    if (heavy[v] != -1)
        decompose(heavy[v], h, adj);
    for (int c : adj[v]) {
        if (c != parent[v] && c != heavy[v])
            decompose(c, c, adj);
    }
}

//this assume 0 base index and root is at 0
void init(vector<vector<int>> const& adj) {
    int n = adj.size();
    parent = vector<int>(n);
    depth = vector<int>(n);
    heavy = vector<int>(n, -1);
    head = vector<int>(n);
    pos = vector<int>(n);
    cur_pos = 0;

    dfs(0, adj);
    decompose(0, 0, adj);
}

//TODO: implement segment tree
//this implementation already handle LCA implicitly
//Query the sum, maximum, minimum, or any other operation that satisfies
//the associative property along the path from node a to node b
//this query function is an example for maximum query

//so make sure to make the correct exclusion in query function below
//if it's query path something on node value then it's fine
//but if it qeury path something value on edge then I normally exclude the right bound
//since I define paths as vertext disjoint paths which excluding first vertext
ll query(int a, int b) { //query sum of edges weight on path version
    ll res = 0;
    for (; head[a] != head[b]; b = parent[head[b]]) {
        if (depth[head[a]] > depth[head[b]])
            swap(a, b);
        ll cur_heavy_path = segment_tree_query(pos[head[b]], pos[b]);
        res += cur_heavy_path;
    }

    if (depth[a] > depth[b])
        swap(a, b);
    ll last_heavy_path = segment_tree_query(pos[a]+1, pos[b]);
    res += last_heavy_path;
    return res;
}
```

divide-and-conquer-dp.h

108b88, 55 lines

```
//TODO: initialize dp , initialize cost() function of use slide()
//ll dp[N][M];

//generic implementation for sliding range technique for logn cost()
//(persistent segtree alternative)
ll ccost = 0;
int cl = 0, cr = -1;
void slide(int l, int r){
    while(cr < r){
        ++cr;
        //add();
        //...
    }
    while(cl > l){
        --cl;
        //add();
        //...
    }
    while(cr > r){
        //remove();
        //...
        --cr;
    }
    while(cl < l){
        //remove();
        //...
        ++cl;
    }
}

void compute(int l, int r, int optl, int optr, int j){
    if(l>r)return;

    int mid = (l+r)>>1;
    //pair<ll, int> best = {0,-1};
    //pair<ll, int> best = {LINF,-1};

    //dp is satisfy quadrangle IE if cost() satisfy qudrangle IE
    //if cost() is QF => opt() is nondecreasing
    for(int k=optl;k<=min(mid,optr);++k){
        slide(k,mid);
        //best = max(best, {((k>0)?dp[k-1][j-1]:0) + ccost, k});
        //best = min(best, {((k>0)?dp[k-1][j-1]:0) + ccost, k});
    }

    //dp[mid][j] = max(dp[mid][j], best.first);
    //dp[mid][j] = min(dp[mid][j], best.first);
    int opt = best.second;
    if(l!=r){
        compute(l,mid-1,optl,opt,j);
        compute(mid+1,r,opt,optr,j);
    }
}

//TODO: set dp to LINF or -LINF
```

cellul4r (3)

3.1 Sorting and Searching from CSES

Technique map

The most important decision is which *shape* the search space has:

- **Sort + scan:** For multiset properties. (Distinct Numbers, Restaurant Customers)
- **Two pointers:** For monotonic boundaries. (Sum of Two Values, Apartments)
- **Binary search on answer:** For monotone predicates. (Factory Machines)

- **Dynamic sorted set:** For predecessor/successor queries. (Concert Tickets, Traffic Lights)
- **Prefix sums + ordered structure:** For subarray constraints. (Subarray Sums)

Sum of Two / Three / Four Values

Problem. Find distinct positions whose values sum to x.

Keep indices before sorting. Two-sum is sorted two-pointers; three-sum fixes one; four-sum uses a map.

```
// two values
sort (value,index)
i=0, j=n-1
while i<j:
   s=a[i]+a[j]
   if s==x: answer
   if s<x: i++ else j--

// four values
for i=0..n-1:
   for j=i+1..n-1:
      if x-a[i]-a[j] in oldPairs: answer
   for j=0..i-1:
      oldPairs[a[i]+a[j]]=(i,j)
```

Solution. Two-sum uses a two-pointer sweep on sorted data. Four-sum stores pair sums in a hash map, adding to the map only indices $< i$ to ensure disjointness.

Analysis. Staggering insertion in four-sum avoids overlapping indices. Fixing an element for three-sum reduces it to two-sum.

Playlist

Problem. Find the longest contiguous segment of distinct songs.

Store the last position of each song and move the left boundary past duplicates.

```
l=0
for r=0..n-1:
   if seen a[r]: l=max(l,last[a[r]]+1)
   last[a[r]]=r
   ans=max(ans,r-l+1)
```

Solution. A sliding window $[l, r]$ avoids duplicates by jumping l past the last seen index of $a[r]$. $O(n)$ time using a hash map.

Analysis. The `max` ensures l never moves backward when an old duplicate is found outside the current window.

Concert Tickets

Problem. Sell the most expensive affordable ticket to each customer in order.

Sell the largest ticket not exceeding each customer's price.

```
multiset tickets
for x in customers:
   it = tickets.upper_bound(x)
   if it == begin: print -1
   else:
      --it
      print *it
      erase it
```

Solution. Use a multiset to query the predecessor of the customer's budget. Greedily taking the largest affordable ticket saves cheaper tickets. $O((n + m) \log n)$ time.

Analysis. Predecessor queries with deletion are perfect for balanced BSTs (multisets).

Restaurant Customers

Problem. Find the maximum overlapping customer intervals.

Sweep arrival/leave events to find maximum overlap.

```
events += (arrival,+1), (leave,-1)
sort by time; process leave before arrival on tie
cur=ans=0
for event: cur += delta, ans=max(ans,cur)
```

Solution. Event sweep converts interval overlap into a prefix sum. Sort by time and track the running total. $O(n \log n)$ time.

Analysis. Processing departures before arrivals on ties handles CSES's non-overlapping endpoint semantics correctly.

Stick Lengths

Problem. Change n stick lengths to be equal, minimizing $\sum |a_i - x|$.

The best target length is the median.

```
sort a
m=a[n/2]
ans=sum(abs(x-m))
```

Solution. The median minimizes the sum of absolute differences. $O(n \log n)$ time.

Analysis. The cost function is convex. Moving the target shifts the penalty equally between left and right elements, zeroing out at the median.

Towers

Problem. Stack cubes. New cubes must be strictly smaller than the tower top. Minimize towers.

Patience sorting: replace the smallest tower top strictly greater than x.

```
multiset tops
for x in cubes:
   it=tops.upper_bound(x)
   if it exists: erase it
   tops.insert(x)
answer=tops.size()
```

Solution. Greedily placing a cube on the tightest valid tower top preserves larger tops for future cubes. $O(n \log n)$ time.

Analysis. This corresponds to patience sorting. By Dilworth's theorem, towers form the longest non-decreasing subsequence.

Traffic Lights

Problem. Add lights to a street and report the longest gap after each addition.

Maintain light positions and segment lengths.

```
pos={0,x}, len={x}
for p in insertions:
   r=*pos.upper_bound(p), l=*prev(...)
   erase len r-l
   insert len p-l and r-p
   pos.insert(p)
   print max(len)
```

Solution. A set tracks positions and a multiset tracks gap lengths. Inserting a light splits one gap into two. $O(n \log n)$ time.

Analysis. Tracking both endpoints and gap lengths explicitly allows $O(\log n)$ updates and maximum queries.

Factory Machines

Problem. Find the minimum time for n machines to make k products.

Binary search the minimum time.

```
ok(T):
   made=0
   for machine in a:
      made += T/machine
      if made >= need: return true
   return false

lo=0, hi=1e18
while lo<hi:
   mid=(lo+hi)/2
   if ok(mid): hi=mid
   else: lo=mid+1
```

Solution. Production capacity is monotonic with time. Binary search the answer up to 10^{18}. $O(n \log(\text{max time}))$ time.

Analysis. The early exit inside `ok()` prevents integer overflow when testing large T.

Array Division

Problem. Divide array into k contiguous subarrays to minimize maximum sum.

Minimize maximum segment sum using binary search + greedy feasibility.

```
ok(limit):
   groups=1, cur=0
   for x in a:
      if x>limit: return false
      if cur+x>limit:
         groups++, cur=0
      cur += x
   return groups <= k
```

Solution. Binary search the maximum allowed sum. The greedy check forms the fewest groups by packing subarrays as tightly as possible. $O(n \log(\sum a_i))$ time.

Analysis. The monotonic search space ranges from $\max(a_i)$ to $\sum a_i$.

Subarray Sums and Maximum Subarray Sum II

Problem. Count subarrays with exact sum (Sums I/II), or find max subarray sum within length bounds.

Positive values allow sliding window. Arbitrary values require prefix sums.

```
// Subarray Sums II
cnt[0]=1
pref=ans=0
for x in a:
   pref += x
   ans += cnt[pref-target]
   cnt[pref]++

// max sum with length [A,B]
insert pref[r-A]
erase pref[r-B-1]
ans=max(ans, pref[r]-min(active))
```

Solution. For exact sum, hash map counts past prefix sums matching the needed difference. For bounded max sum, a multiset queries the minimum valid left prefix.

Analysis. Prefix sums convert subarray constraints into differences between two points, reducing to point lookups or sliding window extremum queries.

3.2 Greedy from CSES

Proof patterns

Greedy is "take what can be proved safe". The proof usually appears as:

- **Exchange argument:** show swapping a non-greedy choice with a greedy one improves or maintains optimality.
- **Stays-ahead:** prove greedy has at least as much progress as any competing solution at each step.
- **Invariant frontier:** maintain a summary state (e.g. "smallest unproducible sum") preserved after each greedy step.
- **Adjacent swap:** show swapping two neighboring elements not in greedy order improves the objective.

Movie Festival

Problem. Given n movies with start/end times ($1 \le n \le 2 \cdot 10^5$), find the max non-overlapping movies you can watch. Example: $[1,3],[2,5],[3,6] \implies 2$.

Select maximum non-overlapping intervals by earliest finishing time.

```
sort movies by end
last=-INF, ans=0
for (l,r) in movies:
   if l >= last:
      ans++, last=r
```

Solution. Sorting by end time leaves the most room for future movies. The exchange argument proves greedy optimality: any later-ending movie can be replaced by the greedy choice. $O(n \log n)$ time, $O(1)$ space.

Analysis. If an optimal solution picks a later-ending first movie, replacing it with our greedy choice is still valid and frees up more time.

Ferris Wheel

Problem. Pair n children into gondolas of capacity x. Max 2 per gondola. Minimize gondolas. Example: weights $[3,5,2,3]$, $x = 6 \implies 2$.

Pair the heaviest remaining child with the lightest if possible.

```
sort w
i=0, j=n-1, ans=0
while i<=j:
   if w[i]+w[j] <= x: i++, j--
   else: j--
   ans++
```

Solution. The heaviest child must ride. If they can pair with the lightest, doing so optimally uses capacity. If they cannot, they ride alone. $O(n \log n)$ time.

Analysis. If the lightest child cannot pair with the heaviest, no one can. If they can, pairing them is optimal because the lightest child is the cheapest partner.

Apartments

Problem. Match n applicants to m apartments within size tolerance k. Maximize matches.

Sort applicants and apartments; discard impossible ones.

```
sort a,b
i=j=ans=0
while i<n and j<m:
   if abs(a[i]-b[j])<=k: ans++, i++, j++
   else if b[j] < a[i]-k: j++
   else: i++
```

Solution. Two-pointer scan on sorted arrays. If an apartment is too small for the current applicant, it's too small for all later ones (skip apartment). If too large, the applicant can't match any remaining (skip applicant). $O(n \log n)$ time.

Analysis. Discarding is safe because arrays are sorted. When a match exists, taking the smallest available apartment greedily is optimal by exchange argument.

Tasks and Deadlines

Problem. n tasks with duration t_i and deadline d_i. Reward is $d_i -$ completion time. Maximize total reward.

Reward is $\sum d_i - \sum C_i$, so sort by duration.

```
sort by duration
time=score=0
for (t,d) in tasks:
   time += t
   score += d-time
```

Solution. $\sum d_i$ is constant, so minimize sum of completion times by doing shortest tasks first. $O(n \log n)$ time.

Analysis. Swapping adjacent tasks A and B (where A is longer) reduces overall completion time by $A - B > 0$. Thus, sorting by duration minimizes the penalty.

Missing Coin Sum

Problem. Find the smallest positive integer sum not formable by a subset of n coins. Example: $[1,1,2,5] \implies 10$.

Maintain $x =$ smallest sum not known reachable.

```
sort coins
x=1
for c in coins:
   if c>x: break
   x += c
answer=x
```

Solution. Sort coins. If we can form all sums up to $x-1$, and the next coin is $c \le x$, we can now form up to $x+c-1$. If $c > x$, x is unreachable. $O(n \log n)$ time.

Analysis. The invariant frontier x represents our unbroken prefix of sums. A coin larger than x leaves a gap at exactly x, halting the algorithm.

Reading Books

Problem. Two people read n books. Book i takes t_i time. Both read all books, but not the same one simultaneously. Minimize total time.

```
answer = max(sum(all times), 2*max_time)
```

Solution. Minimum time is bounded by the total sum (each reads everything) and 2× max book time (one waits while the other reads the longest book). $\max(S, 2 \cdot t_{\max})$ is always achievable. $O(n)$ time.

Analysis. The longest book bottleneck happens when it exceeds all other books combined. Otherwise, they can perfectly overlap their reading schedules.

Movie Festival II

Problem. Generalization of Movie Festival: k club members. Maximize total movies watched.

Assign each movie to the watcher whose ending time is latest but compatible.

```
sort movies by end
ends = multiset of k zeros
for (l,r) in movies:
   it = last end <= l
   if exists:
      erase it
      insert r
      ans++
```

Solution. Sort movies by end time. Maintain k watchers' end times. Assign the movie to the watcher with the largest end time $\leq$ movie start. $O(n \log n)$ time.

Analysis. Using the "tightest fit" watcher preserves the more flexible (earlier ending) watchers for future movies. This greedy choice avoids wasting slack time.

3.3 Dynamic Programming from CSES

How to design the state

DP is useful when choices interact, but the future only needs a compressed summary of the past. A good test: can you compute the answer for the current step using only the state variables, without looking back at individual past choices?

- **Prefix DP:** sums, numbers, rows, columns. (Dice Combinations)
- **Boundary DP:** remember the last value/character/cell. (Array Description)
- **Interval DP:** process a subarray by increasing length. (Removal Game)
- **Subset/profile DP:** bitmask over a small set. (Elevator Rides)
- **Digit DP:** build a number under an upper bound. (Counting Numbers)

Dice Combinations

Problem. Find ordered ways to sum to n using dice throws (1 to 6). Example: $n = 3 \implies 4$.

State. $dp[x]$ = number of ordered dice sequences with sum x.

Transition. Choose the last die roll $d \in [1, 6]$: add $dp[x - d]$.

```
dp[0]=1
for x=1..n:
   for d=1..6:
      if x>=d: dp[x]+=dp[x-d]
```

Solution. Sum all valid ways to reach the previous state $x - d$. $O(n)$ time, $O(n)$ space.

Analysis. This is a basic 1D prefix DP, identical to counting compositions of n into parts from $\{1..6\}$.

Minimizing Coins

Problem. Min coins of given denominations to make sum x. Example: $\{1, 5, 7\}, 11 \implies 3$.

State. $dp[s]$ = minimum coins needed for sum s.

Transition. Last coin is c, so previous sum is $s - c$.

```
dp[0]=0, dp[1..x]=INF
for s=1..x:
   for c in coins:
      if s>=c:
         dp[s]=min(dp[s], dp[s-c]+1)
```

Solution. Try every coin as the last coin used. Take the minimum over valid coins. $O(nx)$ time.

Analysis. A greedy approach (largest coin) fails generally. This DP is equivalent to BFS on an implicit graph of sums.

Coin Combinations I vs II

Problem. I: Count ordered ways to make sum x. **II:** Count unordered ways (multisets).

State. $dp[s]$ counts ways to make sum s.

```
// I: ordered
dp[0]=1
for s=1..x:
   for c in coins:
      if s>=c: dp[s]+=dp[s-c]

// II: unordered
dp[0]=1
for c in coins:
   for s=c..x:
      dp[s]+=dp[s-c]
```

Solution. In I (ordered), outer loop is sums, deciding the last coin per sequence. In II (unordered), outer loop is coins, ensuring each multiset is generated in a fixed coin order. $O(nx)$ time.

Analysis. Loop order determines permutation vs. combination counting. I counts compositions; II counts partitions.

Removing Digits

Problem. Find min steps to reduce n to 0 by subtracting one of its nonzero digits.

State. $dp[x]$ = minimum moves to reduce x to 0.

Transition. Pick any nonzero digit d of x, move to $x - d$.

```
dp[0]=0
for x=1..n:
   dp[x]=INF
   for d in digits(x):
      if d>0: dp[x]=min(dp[x],dp[x-d]+1)
```

Solution. For each number x, extract its digits and subtract each nonzero digit to transition from $x - d$. $O(n \cdot 10)$ time.

Analysis. Transitions go to strictly smaller numbers, so bottom-up works naturally.

Grid Paths I

Problem. Count paths from top-left to bottom-right of an $n \times n$ grid moving right/down, avoiding obstacles.

State. $dp[i][j]$ = number of ways to reach cell (i, j).

Transition. Final move came from up or left, unless blocked.

```
dp[0][0]=1 if open
for i=0..n-1:
   for j=0..n-1:
      if blocked: dp[i][j]=0
      else:
         if i: dp[i][j]+=dp[i-1][j]
         if j: dp[i][j]+=dp[i][j-1]
```

Solution. Paths to (i, j) sum from $(i-1, j)$ and $(i, j-1)$. Blocked cells have 0 paths. $O(n^2)$ time.

Analysis. Basic 2D grid DP counting lattice paths around obstacles.

Book Shop

Problem. Knapsack: maximize pages from n books within budget x. Max 1 copy per book.

State. $dp[m]$ = max pages using processed books and money m.

Transition. Take or skip each book.

```
dp[0..x]=0
for book (price,pages):
   for m=x down to price:
      dp[m]=max(dp[m],dp[m-price]+pages)
```

Solution. 0/1 knapsack. Try including each book. The descending money loop ensures each book is used at most once. $O(nx)$ time.

Analysis. Ascending loop allows unbounded copies. Descending guarantees 0/1 usage by reading from previous states.

Array Description

Problem. Count arrays where adjacent elements differ by at most 1. Some elements are fixed.

State. $dp[i][v]$ = ways for prefix $1..i$ ending with value v.

Transition. Previous value can be $v-1, v, v+1$.

```
init i=1 from fixed/free a[1]
for i=2..n:
   for v=1..m:
      if a[i] fixed and a[i]!=v: continue
      dp[i][v]=dp[i-1][v-1]+dp[i-1][v]+dp[i-1][v+1]
```

Solution. A prefix is fully summarized by its last value. Sum transitions from adjacent allowed values. $O(nm)$ time.

Analysis. Classic boundary DP pattern since constraints are only on adjacent pairs.

Counting Towers

Problem. Count $2 \times n$ grid tilings with 1×2 and 2×1 blocks extending upward.

State. Small automaton: joined top block (a) or separated top blocks (b).

Transition. Extend the tower one row upward.

```
a[1]=1, b[1]=1
for h=2..MAX:
   a[h]=2*a[h-1]+b[h-1]
   b[h]=a[h-1]+4*b[h-1]
answer(n)=a[n]+b[n]
```

Solution. A finite state machine based on the top boundary shape. The coefficients encode valid extensions. $O(N_{max})$ time.

Analysis. Classify only the boundary shape that affects the next row.

Edit Distance

Problem. Min inserts, deletes, or replaces to change string a into b.

State. $dp[i][j]$ = min ops for first i of a into first j of b.

Transition. Delete, insert, replace/match.

```
dp[i][0]=i, dp[0][j]=j
for i=1..n:
   for j=1..m:
      cost = (a[i-1]!=b[j-1])
      dp[i][j]=min(dp[i-1][j]+1,
                   dp[i][j-1]+1,
                   dp[i-1][j-1]+cost)
```

Solution. Three branches for the edit choices. Match is free, mismatch costs 1. $O(nm)$ time.

Analysis. Standard 2D sequence alignment DP (Levenshtein distance).

Rectangle Cutting

Problem. Min cuts to split $a \times b$ rectangle into squares.

State. $dp[a][b]$ = minimum cuts to split $a \times b$.

Transition. Try every vertical and horizontal first cut.

```
for a=1..A:
  for b=1..B:
    if a==b: dp[a][b]=0
    else:
      dp[a][b]=INF
      for x=1..a-1: dp[a][b]=min(dp[a][b],1+dp[x][b]+dp[a-x][b])
      for y=1..b-1: dp[a][b]=min(dp[a][b],1+dp[a][y]+dp[a][b-y])
```

Solution. Try all splitting lines and recursively solve the pieces. $O(ab(a+b))$ time.

Analysis. 2D interval-like DP, analogous to optimal matrix chain multiplication.

Money Sums

Problem. Find all achievable subset sums from n coins.

State. $possible[s]$ = is sum s achievable?

Transition. Take it or skip it.

```
possible[0]=true
for coin in coins:
   for s=MAX down to coin:
      possible[s] |= possible[s-coin]
```

Solution. 0/1 knapsack decision DP. Descending loop ensures at most 1 usage per coin. $O(nS)$ time.

Analysis. Pseudo-polynomial subset sum. Bitset optimizes this beautifully.

Removal Game

Problem. 2-player game removing ends of an array. Maximize score difference.

State. $dp[l][r]$ = max score diff current player can force on $l..r$.

Transition. Take left or right, opponent plays remainder.

```
dp[i][i]=a[i]
for len=2..n:
   for l=0..n-len:
      r=l+len-1
      dp[l][r]=max(a[l]-dp[l+1][r],
                   a[r]-dp[l][r-1])
answer=(sum+dp[0][n-1])/2
```

Solution. Interval DP. The difference achieved is the taken item minus the opponent's best response on the remainder. $O(n^2)$ time.

Analysis. The "difference game" trick reduces two scores to one difference state.

Two Sets II

Problem. Count ways to partition $1..n$ into two equal-sum subsets.

State. Count subsets with sum $S = n(n+1)/4$.

Transition. 0/1 subset sum over numbers $1..n$.

```
if n*(n+1)/2 odd: answer=0
target=sum/2
dp[0]=1
for x=1..n:
   for s=target down to x:
      dp[s]+=dp[s-x]
answer=dp[target] * inv2
```

Solution. 0/1 knapsack to target sum. Divide by 2 (modular inverse) since each partition is generated twice. $O(n^3)$ time.

Analysis. Simple subset sum with a symmetry argument.

Projects

Problem. Weighted interval scheduling: maximize reward from non-overlapping intervals.

State. $dp[i]$ = max reward using first i projects sorted by end time.

Transition. Skip i, or take i plus best compatible prefix.

```
sort by end
for i=1..n:
   p = last project with end[p] < start[i]
   dp[i]=max(dp[i-1], dp[p]+reward[i])
```

Solution. Binary search the last project ending before current project starts. $O(n \log n)$ time.

Analysis. Requires DP since high-weight intervals break greedy scheduling.

Elevator Rides

Problem. Min rides of capacity x to transport n people ($n \leq 20$).

State. $dp[mask] = (rides, lastWeight)$ = best packing for selected mask.

Transition. Remove one person and add to previous best.

```
dp[0]=(1,0)
for mask=1..(1<<n)-1:
   dp[mask]=(INF,INF)
   for i in mask:
      cand=dp[mask without i]
      if cand.weight+w[i] <= X: cand.weight += w[i]
      else: cand.rides++, cand.weight=w[i]
      dp[mask]=min_lex(dp[mask],cand)
```

Solution. Subset DP. Lexicographically minimizing '(rides, lastWeight)' leaves max room in the current ride. $O(n2^n)$ time.

Analysis. Keeping the last ride as empty as possible is a greedy-optimal substructure within the DP mask.

Counting Tilings

Problem. Count ways to tile $n \times m$ grid with 1×2 and 2×1 dominoes.

State. $dp[col][mask]$ = ways to reach column 'col' with 'mask' protruding.

Transition. Fill current column by DFS.

```
gen(row, mask, nextMask):
   if row==n: add dp[col][mask] to dp[col+1][nextMask]
   else if mask has row: gen(row+1,mask,nextMask)
   else:
      place horizontal -> nextMask has row
      if row+1 free: place vertical in current column
```

Solution. Profile DP. DFS generates valid vertical/horizontal placements transitioning between column boundaries. $O(m4^n)$ time.

Analysis. Standard state-compression DP for grids with one small dimension.

Counting Numbers

Problem. Count integers in $[a, b]$ with no adjacent equal digits.

State. $f(pos, prev, tight, started)$ building left to right.

Transition. Try allowed digits, rejecting equals after start.

```
solve(pos, prev, tight, started):
   if pos==len: return 1
   limit = tight ? digit[pos] : 9
   ans=0
   for d=0..limit:
      if started and d==prev: continue
      ans += solve(pos+1,d,tight&&d==limit,started||d>0)
```

Solution. Digit DP with state tracking the upper bound ('tight') and leading zeros ('started'). O(digits) time.

Analysis. Computes $f(b) - f(a-1)$ to find the answer for the range.

Increasing Subsequence

Problem. Length of longest strictly increasing subsequence.

State. $tail[len]$ = smallest ending value of a length-len LIS.

```
tail = empty
for x in a:
   it = lower_bound(tail,x)
   if it==end: push x
   else *it=x
answer=tail.size()
```

Solution. Binary search in the monotonic $tail$ array and replace the smallest larger element. $O(n \log n)$ time.

Analysis. This is patience sorting. Smaller $tail$ values preserve flexibility for future extensions.

3.4 Graph Algorithms from CSES

Algorithm map

First classify the graph question, then pick the tool:

- **Reachability:** DFS/BFS.
- **Unweighted shortest path:** BFS + parent array.
- **Nonnegative weighted shortest path:** Dijkstra.
- **Negative edges/cycles:** Bellman-Ford.
- **All-pairs small graph:** Floyd-Warshall.
- **DAG:** topological sort + DP.
- **Mutual reachability:** SCC (Kosaraju or Tarjan).
- **Online connectivity:** DSU.
- **Capacity/matching:** Max flow (Dinic's).

Graph representation

```
vector<int> adj[N]; // unweighted
vector<pair<int,ll>> adj[N]; // weighted
vector<tuple<int,int,ll>> edges; // Bellman/Kruskal
```

Adjacency list is best. Adjacency matrices are only for Floyd-Warshall.

Counting Rooms

Problem. Count connected open components in an $n \times m$ grid.

State. Grid cells are graph nodes; open neighboring cells have edges.

```
ans=0
for each cell:
   if open and unseen:
      ans++
      BFS/DFS mark all reachable open cells
```

Solution. Iterate over cells. When finding an unseen open cell, increment count and flood-fill to mark the whole room. $O(nm)$ time.

Analysis. Connected components on an implicit grid graph.

Labyrinth and Message Route

Problem. Find shortest path between two points on an unweighted grid/graph.

State. BFS distance and parent from the start.

```
q.push(start), seen[start]=true
while q not empty:
   v=q.pop()
   for u in adj[v]:
      if not seen[u]:
         seen[u]=true
         par[u]=v
         q.push(u)
if target seen: reconstruct by parent
```

Solution. BFS explores level by level, ensuring the first time a node is reached is via the shortest path. $O(V+E)$ time.

Analysis. Parent arrays encode the shortest-path tree for path reconstruction.

Building Roads

Problem. Find the minimum new roads to connect all n cities.

State. Components of an undirected graph.

```
leaders=[]
for v=1..n:
   if unseen[v]:
      leaders.push(v)
      DFS(v)
print leaders.size()-1
connect leaders[0] to leaders[i]
```

Solution. Find all connected components and pick a leader for each. Chain all leaders together. $O(n+m)$ time.

Analysis. c components require exactly $c-1$ edges to become fully connected.

Building Teams

Problem. Divide pupils into two teams with no intra-team friendships (bipartite check).

State. Color each node 1 or 2.

```
for each component:
   color[start]=1
   BFS:
      for u in adj[v]:
         if color[u]==0: color[u]=3-color[v]
         else if color[u]==color[v]: impossible
```

Solution. BFS layers alternate colors. If an edge connects same-color nodes, it's not bipartite. $O(n+m)$ time.

Analysis. Bipartite checking fails exactly when an odd-length cycle exists.

Round Trip

Problem. Find a cycle of length ≥ 3 in an undirected graph.

State. DFS parent tree in an undirected graph.

```
dfs(v,p):
   seen[v]=true
   for u in adj[v]:
      if u==p: continue
      if seen[u]: recover cycle v..u using parents
      else par[u]=v, dfs(u,v)
```

Solution. Run DFS. If we hit a visited node that isn't the parent, we found a back-edge (cycle). $O(n+m)$ time.

Analysis. Any back-edge in a DFS tree defines a fundamental cycle.

Monsters

Problem. Escape a grid. You and monsters move 1 cell per second. Can you reach the boundary?

State. Monster arrival time and player arrival time.

```
multi-source BFS from all monsters -> mdist
BFS from A:
   for neighbor u:
      nt = pdist[v]+1
      if open and nt < mdist[u]:
         visit u
   if boundary reached: escape
```

Solution. Multi-source BFS computes earliest monster arrival per cell. Player BFS only steps on cells where player arrives strictly before any monster. $O(nm)$ time.

Analysis. Precomputing monster times converts dynamic enemies into static cell constraints.

Shortest Routes I

Problem. Shortest path from node 1 with nonnegative edge weights.

State. $dist[v]$ shortest known distance from 1.

```
dist[1]=0
pq.push(0,1)
while pq not empty:
   (d,v)=pop min
   if d!=dist[v]: continue
   for (u,w) in adj[v]:
      if d+w < dist[u]:
         dist[u]=d+w
         pq.push(dist[u],u)
```

Solution. Dijkstra's algorithm. Processes nodes by increasing distance using a min-heap. $O((n+m)\log n)$ time.

Analysis. The greedy choice is optimal since no negative edges can provide a cheaper route later.

Shortest Routes II

Problem. All-pairs shortest paths on a small graph ($n \leq 500$).

State. $dist[i][j]$ shortest path between all pairs.

```
dist[i][i]=0
for edge a-b weight w:
   dist[a][b]=dist[b][a]=min(...,w)
for k=1..n:
  for i=1..n:
    for j=1..n:
     dist[i][j]=min(dist[i][j],dist[i][k]+dist[k][j])
```

Solution. Floyd-Warshall algorithm. Incrementally allows nodes $1..k$ as intermediate stops. $O(n^3)$ time.

Analysis. The outermost loop must be k. State transition is based on path concatenation.

High Score and Cycle Finding

Problem. Max weight path, or detect negative cycles.

State. Bellman-Ford distances. High Score can be solved by negating rewards.

```
dist[1]=0
repeat n-1:
   relax all edges
mark nodes relaxed on nth round
if a relaxed node can reach n: answer is infinite
```

Solution. Bellman-Ford computes shortest paths. A relaxation on the n-th pass indicates a cycle. $O(nm)$ time.

Analysis. For high score, a positive cycle matters only if it's reachable from start and can reach the end.

Flight Discount

Problem. Min cost path, where one edge's cost can be halved.

State. $dist[v][0/1]$ = best cost to v before/after using coupon.

```
Dijkstra over (node,used)
for edge v->u weight w:
   relax (u,used) by +w
   if used==0:
      relax (u,1) by +w/2
```

Solution. Run Dijkstra on a state-expanded graph with $2n$ nodes. $O((n+m)\log n)$ time.

Analysis. Expands state space to track resource usage.

Course Schedule

Problem. Topological sort of n courses with prerequisite edges.

State. In-degree counts remaining prerequisites.

```
push all indegree-0 nodes
while queue not empty:
   v=pop, order.push(v)
   for u in adj[v]:
      if --indeg[u]==0: push u
if order.size()<n: IMPOSSIBLE
```

Solution. Kahn's algorithm. Process nodes with 0 in-degree, decrementing neighbors. $O(n+m)$ time.

Analysis. If all nodes aren't processed, a cycle exists.

Longest Flight Route

Problem. Longest path in a DAG.

State. $dp[v]$ = max number of nodes on a path from 1 to v in a DAG.

```
toposort
dp[1]=1, others=-INF
for v in order:
   for u in adj[v]:
      if dp[v]+1 > dp[u]:
         dp[u]=dp[v]+1
         par[u]=v
```

Solution. Process nodes in topological order, relaxing edges to maximize path length. $O(n+m)$ time.

Analysis. DAG property allows DP instead of NP-hard general longest path.

Game Routes

Problem. Count paths in a DAG from node 1 to n.

State. $dp[v]$ = number of paths from 1 to v.

```
toposort
dp[1]=1
for v in order:
   for u in adj[v]:
      dp[u]=(dp[u]+dp[v])%MOD
```

Solution. Process in topological order, pushing path counts forward. $O(n+m)$ time.

Analysis. Standard DAG path-counting DP.

Investigation

Problem. Find min cost, number of min cost routes, min flights, max flights.

State. For each city: shortest distance, number of shortest routes, min flights, max flights.

```
if nd < dist[u]:
   dist[u]=nd; ways[u]=ways[v]; mn[u]=mn[v]+1; mx[u]=mx[v]+1
else if nd == dist[u]:
   ways[u]+=ways[v]; mn[u]=min(mn[u],mn[v]+1); mx[u]=max(mx[u],mx[v]+1)
```

Solution. Run Dijkstra, but track all 4 metrics. Merge metrics on equal distances. $O((n+m)\log n)$ time.

Analysis. Aggregating path stats over shortest-path DAG.

Planets Queries I and II

Problem. Navigate a functional graph.

```
up[0][v]=next[v]
for j=1..LOG:
   up[j][v]=up[j-1][ up[j-1][v] ]
jump(v,k):
   for bit j: if k has bit: v=up[j][v]
```

Solution. Binary lifting computes k-th successor in $O(\log k)$ time.

Analysis. Pure binary lifting on a functional graph.

Planets Cycles

Problem. Find path length until a cycle repeats in a functional graph.

State. Functional graph components consist of a directed cycle with trees feeding into it.

```
find cycle nodes by indegree pruning or DFS states
set answer for cycle nodes = cycle length
reverse edges DFS from cycle:
   ans[child] = ans[parent] + 1
```

Solution. Identify cycles, then propagate distances outwards through reversed trees. $O(n)$ time.

Analysis. "Rho-shaped" components map perfectly to this decomposition.

Planets and Kingdoms

Problem. Find Strongly Connected Components (SCCs).

State. Strongly connected components.

```
DFS original graph, push by finish time
DFS reversed graph in reverse finish order
each DFS tree is one SCC
```

Solution. Kosaraju's algorithm finds SCCs by running DFS twice (once on reversed graph). $O(n+m)$ time.

Analysis. Compresses cycles to form a DAG of SCCs.

Giant Pizza

Problem. 2-SAT problem for pizza toppings.

```
(a OR b) -> (!a -> b), (!b -> a)
find SCCs
if x and !x in same SCC: impossible
assignment by reverse SCC order
```

Solution. Convert to implication graph, find SCCs. If variable and negation share an SCC, unsatisfiable. $O(n+m)$ time.

Analysis. Reverse topological order of SCCs provides a valid truth assignment.

Road Reparation

Problem. Minimum Spanning Tree (MST).

```
sort edges by weight
for edge (w,a,b):
   if find(a)!=find(b):
      unite(a,b)
      cost += w
```

Solution. Kruskal's algorithm adds edges greedily avoiding cycles via DSU. $O(m\log m)$ time.

Analysis. The cut property guarantees greedy choice works.

Road Construction

Problem. Maintain component count and max size as edges are added.

State. DSU component count and largest component size after each edge.

```
components=n, largest=1
for road a-b:
   if unite(a,b):
      components--
      largest=max(largest,size[root])
   print components, largest
```

Solution. DSU tracks components and sizes online. $O(m\alpha(n))$ time.

Analysis. Perfect fit for disjoint set union since edges are only added.

Mail Delivery

Problem. Find an Eulerian circuit.

```
if any degree odd or disconnected: IMPOSSIBLE
Hierholzer(v):
   while unused edge from v:
      mark edge used
      Hierholzer(to)
   path.push_back(v)
```

Solution. Hierholzer's algorithm traces circuits greedily, backtracking to splice them. $O(n+m)$ time.

Analysis. Exists iff all degrees are even and edges are connected.

Download Speed

Problem. Find Maximum Flow from 1 to n.

```
while BFS builds levels:
   reset ptr
   while pushed = DFS(s, INF):
      flow += pushed
```

Solution. Dinic's algorithm pushes flow along shortest paths. $O(n^2m)$ time.

Analysis. Standard max-flow application.

School Dance

Problem. Maximum Bipartite Matching.

```
source -> boys capacity 1
boy -> girl if pair allowed
girls -> sink capacity 1
run maxflow; flow edges boy->girl are matches
```

Solution. Add source/sink with unit capacities. Run max flow. $O(m\sqrt{n})$ time.

Analysis. Solves bipartite matching natively using Dinic's.

Hamiltonian Flights

Problem. Count Hamiltonian paths from 1 to n.

State. $dp[mask][v]$ = ways to visit mask ending at v.

```
dp[1<<0][0]=1
for mask containing 0:
   for v in mask:
      for u with edge v->u and u not in mask:
         dp[mask|1<<u][u]+=dp[mask][v]
answer=dp[(1<<n)-1][n-1]
```

Solution. Held-Karp bitmask DP. $O(2^n n^2)$ time.

Analysis. Tractable because $n \le 20$.

Ohm (4)

Mathematics (5)

5.1 Equations

$$ax^2+bx+c=0 \Rightarrow x=\frac{-b\pm\sqrt{b^2-4ac}}{2a}$$

The extremum is given by $x=-b/2a$.

$$\begin{aligned} ax+by&=e \\ cx+dy&=f \end{aligned} \Rightarrow \begin{aligned} x&=\frac{ed-bf}{ad-bc} \\ y&=\frac{af-ec}{ad-bc} \end{aligned}$$

In general, given an equation $Ax=b$, the solution to a variable x_i is given by

$$x_i=\frac{\det A_i'}{\det A}$$

where A_i' is A with the i'th column replaced by b.

Non-distinct roots r become polynomial factors, e.g. $a_n=(d_1n+d_2)r^n$.

5.2 Trigonometry

$$\sin(v+w)=\sin v\cos w+\cos v\sin w$$
$$\cos(v+w)=\cos v\cos w-\sin v\sin w$$

$$\tan(v+w)=\frac{\tan v+\tan w}{1-\tan v\tan w}$$
$$\sin v+\sin w=2\sin\frac{v+w}{2}\cos\frac{v-w}{2}$$
$$\cos v+\cos w=2\cos\frac{v+w}{2}\cos\frac{v-w}{2}$$
$$(V+W)\tan(v-w)/2=(V-W)\tan(v+w)/2$$

where V,W are lengths of sides opposite angles v,w.

$$a\cos x+b\sin x=r\cos(x-\phi)$$
$$a\sin x+b\cos x=r\sin(x+\phi)$$

where $r=\sqrt{a^2+b^2}, \phi=\text{atan2}(b,a)$.

5.3 Geometry

5.3.1 Triangles

Side lengths: a,b,c

Semiperimeter: $p=\frac{a+b+c}{2}$

Area: $A=\sqrt{p(p-a)(p-b)(p-c)}$

Circumradius: $R=\frac{abc}{4A}$

Inradius: $r=\frac{A}{p}$

Length of median (divides triangle into two equal-area triangles): $m_a=\frac{1}{2}\sqrt{2b^2+2c^2-a^2}$

Length of bisector (divides angles in two):

$$s_a=\sqrt{bc\left[1-\left(\frac{a}{b+c}\right)^2\right]}$$

Law of sines: $\frac{\sin\alpha}{a}=\frac{\sin\beta}{b}=\frac{\sin\gamma}{c}=\frac{1}{2R}$

Law of cosines: $a^2=b^2+c^2-2bc\cos\alpha$

Law of tangents: $\frac{a+b}{a-b}=\frac{\tan\frac{\alpha+\beta}{2}}{\tan\frac{\alpha-\beta}{2}}$

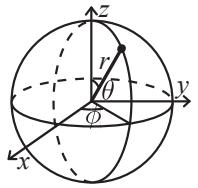
5.3.2 Quadrilaterals

With side lengths a,b,c,d, diagonals e,f, diagonals angle θ, area A and magic flux $F=b^2+d^2-a^2-c^2$:

$$4A=2ef\cdot\sin\theta=F\tan\theta=\sqrt{4e^2f^2-F^2}$$

For cyclic quadrilaterals the sum of opposite angles is $180°$, $ef=ac+bd$, and $A=\sqrt{(p-a)(p-b)(p-c)(p-d)}$.

5.3.3 Spherical coordinates



$$\begin{aligned} x&=r\sin\theta\cos\phi \\ y&=r\sin\theta\sin\phi \\ z&=r\cos\theta \end{aligned} \qquad \begin{aligned} r&=\sqrt{x^2+y^2+z^2} \\ \theta&=\text{acos}(z/\sqrt{x^2+y^2+z^2}) \\ \phi&=\text{atan2}(y,x) \end{aligned}$$

5.4 Derivatives/Integrals

$$\frac{d}{dx}\arcsin x=\frac{1}{\sqrt{1-x^2}} \qquad \frac{d}{dx}\arccos x=-\frac{1}{\sqrt{1-x^2}}$$
$$\frac{d}{dx}\tan x=1+\tan^2 x \qquad \frac{d}{dx}\arctan x=\frac{1}{1+x^2}$$
$$\int\tan ax=-\frac{\ln|\cos ax|}{a} \qquad \int x\sin ax=\frac{\sin ax-ax\cos ax}{a^2}$$
$$\int e^{-x^2}=\frac{\sqrt{\pi}}{2}\text{erf}(x) \qquad \int xe^{ax}dx=\frac{e^{ax}}{a^2}(ax-1)$$

Integration by parts:

$$\int_a^b f(x)g(x)dx=[F(x)g(x)]_a^b-\int_a^b F(x)g'(x)dx$$

5.5 Sums

$$c^a+c^{a+1}+\cdots+c^b=\frac{c^{b+1}-c^a}{c-1}, c\neq 1$$

$$1+2+3+\cdots+n=\frac{n(n+1)}{2}$$
$$1^2+2^2+3^2+\cdots+n^2=\frac{n(2n+1)(n+1)}{6}$$
$$1^3+2^3+3^3+\cdots+n^3=\frac{n^2(n+1)^2}{4}$$
$$1^4+2^4+3^4+\cdots+n^4=\frac{n(n+1)(2n+1)(3n^2+3n-1)}{30}$$

5.6 Series

$$e^x=1+x+\frac{x^2}{2!}+\frac{x^3}{3!}+\ldots,\ (-\infty<x<\infty)$$
$$\ln(1+x)=x-\frac{x^2}{2}+\frac{x^3}{3}-\frac{x^4}{4}+\ldots,\ (-1<x\leq 1)$$
$$\sqrt{1+x}=1+\frac{x}{2}-\frac{x^2}{8}+\frac{2x^3}{32}-\frac{5x^4}{128}+\ldots,\ (-1\leq x\leq 1)$$
$$\sin x=x-\frac{x^3}{3!}+\frac{x^5}{5!}-\frac{x^7}{7!}+\ldots,\ (-\infty<x<\infty)$$
$$\cos x=1-\frac{x^2}{2!}+\frac{x^4}{4!}-\frac{x^6}{6!}+\ldots,\ (-\infty<x<\infty)$$

5.7 Probability theory

Let X be a discrete random variable with probability $p_X(x)$ of assuming the value x. It will then have an expected value (mean) $\mu=\mathbb{E}(X)=\sum_x xp_X(x)$ and variance $\sigma^2=V(X)=\mathbb{E}(X^2)-(\mathbb{E}(X))^2=\sum_x(x-\mathbb{E}(X))^2p_X(x)$ where σ is the standard deviation. If X is instead continuous it will have a probability density function $f_X(x)$ and the sums above will instead be integrals with $p_X(x)$ replaced by $f_X(x)$.

Expectation is linear:

$$\mathbb{E}(aX+bY)=a\mathbb{E}(X)+b\mathbb{E}(Y)$$

For independent X and Y,

$$V(aX+bY)=a^2V(X)+b^2V(Y).$$

5.7.1 Discrete distributions

Binomial distribution

The number of successes in n independent yes/no experiments, each which yields success with probability p is $\mathrm{Bin}(n,p)$, $n=1,2,\ldots,\ 0\leq p\leq 1$.

$$p(k)=\binom{n}{k}p^k(1-p)^{n-k}$$

$$\mu=np,\ \sigma^2=np(1-p)$$

$\mathrm{Bin}(n,p)$ is approximately $\mathrm{Po}(np)$ for small p.

First success distribution

The number of trials needed to get the first success in independent yes/no experiments, each which yields success with probability p is $\mathrm{Fs}(p)$, $0\leq p\leq 1$.

$$p(k)=p(1-p)^{k-1},\ k=1,2,\ldots$$

$$\mu=\frac{1}{p},\ \sigma^2=\frac{1-p}{p^2}$$

Poisson distribution

The number of events occurring in a fixed period of time t if these events occur with a known average rate κ and independently of the time since the last event is $\mathrm{Po}(\lambda)$, $\lambda=t\kappa$.

$$p(k)=e^{-\lambda}\frac{\lambda^k}{k!},k=0,1,2,\ldots$$

$$\mu=\lambda,\ \sigma^2=\lambda$$

5.7.2 Continuous distributions

Uniform distribution

If the probability density function is constant between a and b and 0 elsewhere it is $\mathrm{U}(a,b)$, $a<b$.

$$f(x)=\begin{cases}\frac{1}{b-a} & a<x<b\\ 0 & \text{otherwise}\end{cases}$$

$$\mu=\frac{a+b}{2},\ \sigma^2=\frac{(b-a)^2}{12}$$

Exponential distribution

The time between events in a Poisson process is $\mathrm{Exp}(\lambda)$, $\lambda>0$.

$$f(x)=\begin{cases}\lambda e^{-\lambda x} & x\geq 0\\ 0 & x<0\end{cases}$$

$$\mu=\frac{1}{\lambda},\ \sigma^2=\frac{1}{\lambda^2}$$

Normal distribution

Most real random values with mean μ and variance σ^2 are well described by $\mathcal{N}(\mu,\sigma^2)$, $\sigma>0$.

$$f(x)=\frac{1}{\sqrt{2\pi\sigma^2}}e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

If $X_1\sim\mathcal{N}(\mu_1,\sigma_1^2)$ and $X_2\sim\mathcal{N}(\mu_2,\sigma_2^2)$ then

$$aX_1+bX_2+c\sim\mathcal{N}(\mu_1+\mu_2+c,a^2\sigma_1^2+b^2\sigma_2^2)$$

Chicken McNugget Theorem

The **Chicken McNugget Theorem** (or **Postage Stamp Problem** or **Frobenius Coin Problem**) states that for any two relatively prime positive integers m,n, the greatest integer that cannot be written in the form $am+bn$ for nonnegative integers a,b is $mn-m-n$.

A consequence of the theorem is that there are exactly $\frac{(m-1)(n-1)}{2}$ positive integers which cannot be expressed in the form $am+bn$. The proof is based on the fact that in each pair of the form $(k,mn-m-n-k)$, exactly one element is expressible.

Rearrangement Inequality

The **Rearrangement Inequality** states that, if $A=\{a_1,a_2,\cdots,a_n\}$ is a permutation of a finite set (in fact, multiset) of real numbers and $B=\{b_1,b_2,\cdots,b_n\}$ is a permutation of another finite set of real numbers, the quantity $a_1b_1+a_2b_2+\cdots+a_nb_n$ is maximized when A and B are similarly sorted (that is, if a_k is greater than or equal to exactly i of the other members of A, then b_k is also greater than or equal to exactly i of the other members of B). Conversely, $a_1b_1+a_2b_2+\cdots+a_nb_n$ is minimized when A and B are oppositely sorted (that is, if a_k is less than or equal to exactly i of the other members of A, then b_k is also greater than or equal to exactly i of the other members B).

Wilson's Theorem

In number theory, **Wilson's Theorem** states that if integer $p>1$, then $(p-1)!+1$ is divisible by p if and only if p is prime. It was stated by John Wilson. The French mathematician Lagrange proved it in 1771.

Techniques (A)

techniques.txt 159 lines

```
Recursion
Divide and conquer
  Finding interesting points in N log N
Algorithm analysis
  Master theorem
  Amortized time complexity
Greedy algorithm
  Scheduling
  Max contiguous subvector sum
  Invariants
  Huffman encoding
Graph theory
  Dynamic graphs (extra book-keeping)
  Breadth first search
  Depth first search
  * Normal trees / DFS trees
  Dijkstra's algorithm
  MST: Prim's algorithm
  Bellman-Ford
  Konig's theorem and vertex cover
  Min-cost max flow
  Lovasz toggle
  Matrix tree theorem
  Maximal matching, general graphs
  Hopcroft-Karp
  Hall's marriage theorem
  Graphical sequences
  Floyd-Warshall
  Euler cycles
  Flow networks
  * Augmenting paths
  * Edmonds-Karp
  Bipartite matching
  Min. path cover
  Topological sorting
  Strongly connected components
  2-SAT
  Cut vertices, cut-edges and biconnected components
  Edge coloring
  * Trees
  Vertex coloring
  * Bipartite graphs (=> trees)
  * 3^n (special case of set cover)
  Diameter and centroid
  K'th shortest path
  Shortest cycle
Dynamic programming
  Knapsack
  Coin change
  Longest common subsequence
  Longest increasing subsequence
  Number of paths in a dag
  Shortest path in a dag
  Dynprog over intervals
  Dynprog over subsets
  Dynprog over probabilities
  Dynprog over trees
  3^n set cover
  Divide and conquer
  Knuth optimization
  Convex hull optimizations
  RMQ (sparse table a.k.a 2^k-jumps)
  Bitonic cycle
  Log partitioning (loop over most restricted)
Combinatorics
  Computation of binomial coefficients
  Pigeon-hole principle
  Inclusion/exclusion
  Catalan number
  Pick's theorem
Number theory
  Integer parts
  Divisibility
  Euclidean algorithm
  Modular arithmetic
  * Modular multiplication
  * Modular inverses
  * Modular exponentiation by squaring
  Chinese remainder theorem
  Fermat's little theorem
  Euler's theorem
  Phi function
  Frobenius number
  Quadratic reciprocity
  Pollard-Rho
  Miller-Rabin
  Hensel lifting
  Vieta root jumping
Game theory
  Combinatorial games
  Game trees
  Mini-max
  Nim
  Games on graphs
  Games on graphs with loops
  Grundy numbers
  Bipartite games without repetition
  General games without repetition
  Alpha-beta pruning
Probability theory
Optimization
  Binary search
  Ternary search
  Unimodality and convex functions
  Binary search on derivative
Numerical methods
  Numeric integration
  Newton's method
  Root-finding with binary/ternary search
  Golden section search
Matrices
  Gaussian elimination
  Exponentiation by squaring
Sorting
  Radix sort
Geometry
  Coordinates and vectors
  * Cross product
  * Scalar product
  Convex hull
  Polygon cut
  Closest pair
  Coordinate-compression
  Quadtrees
  KD-trees
  All segment-segment intersection
Sweeping
  Discretization (convert to events and sweep)
  Angle sweeping
  Line sweeping
  Discrete second derivatives
Strings
  Longest common substring
  Palindrome subsequences
  Knuth-Morris-Pratt
  Tries
  Rolling polynomial hashes
  Suffix array
  Suffix tree
  Aho-Corasick
  Manacher's algorithm
  Letter position lists
Combinatorial search
  Meet in the middle
  Brute-force with pruning
  Best-first (A*)
  Bidirectional search
  Iterative deepening DFS / A*
Data structures
  LCA (2^k-jumps in trees in general)
  Pull/push-technique on trees
  Heavy-light decomposition
  Centroid decomposition
  Lazy propagation
  Self-balancing trees
  Convex hull trick (wcipeg.com/wiki/Convex_hull_trick)
  Monotone queues / monotone stacks / sliding queues
  Sliding queue using 2 stacks
  Persistent segment tree
```